ตารางแฮชบน GPU ที่ดีกว่าเดิม

(Better GPU Hash Tables)

Muhammad A. Awad
Dept. of Elect. and Comp. Engineering
UC Davis
Davis, CA, USA
mawad@ucdavis.edu

Saman Ashkiani
Google
Mountain View, CA, USA
sa.ashkiani@gmail.com

Serban D. Porumbescu
Dept. of Elect. and Comp. Engineering
UC Davis
Davis, CA, USA
sdporumbescu@ucdavis.edu

Martín Farach-Colton
Dept. of Computer Science
Rutgers University
Piscataway, NJ, USA
farach@cs.rutgers.edu

John D. Owens
Dept. of Elect. and Comp. Engineering
UC Davis
Davis, CA, USA
jowens@ece.ucdavis.edu
ORCID: 0000-0001-6582-8237

หมายเหตุการแปล: เอกสารนี้เป็น**ฉบับแปลภาษาไทยที่ไม่เป็นทางการ**ของบทความ "Better GPU Hash Tables" โดย Muhammad A. Awad, Saman Ashkiani, Serban D. Porumbescu, Martín Farach-Colton และ John D. Owens (arXiv:2108.07232v3, https://arxiv.org/abs/2108.07232v3) ต้นฉบับเผยแพร่ภายใต้สัญญาอนุญาต Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0, http://creativecommons.org/licenses/by-nc-sa/4.0/) และฉบับแปลนี้เผยแพร่ภายใต้สัญญาอนุญาตเดียวกัน (CC BY-NC-SA 4.0) ฉบับแปลนี้ไม่ได้จัดทำหรือรับรองโดยผู้เขียนต้นฉบับ หากมีข้อสงสัยโปรดยึดต้นฉบับภาษาอังกฤษเป็นหลัก

Translation notice: This document is an **unofficial Thai translation** of "Better GPU Hash Tables" by Muhammad A. Awad, Saman Ashkiani, Serban D. Porumbescu, Martín Farach-Colton, and John D. Owens (arXiv:2108.07232v3, https://arxiv.org/abs/2108.07232v3). The original work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0, http://creativecommons.org/licenses/by-nc-sa/4.0/), and this translation is released under the same license (CC BY-NC-SA 4.0). It is not produced or endorsed by the original authors; in case of doubt, the English original prevails.

***บทคัดย่อ*—เรา ทบทวน ปัญหา การ สร้าง ตา ราง แฮช แบบ สถิต (static hash table) บน GPU อีกครั้ง และออกแบบพร้อมทั้งสร้างตารางแฮชแบบแบ่งเป็นบักเก็ต (bucketed hash table) สามแบบที่ใช้รูปแบบการโพรบ (probing scheme) แตกต่างกัน การพัฒนาของเราเป็นแบบไม่ใช้ล็อก (lock-free) และมีรูปแบบการเข้าถึงหน่วยความจำ (memory access pattern) ที่มีประสิทธิภาพ ดังนั้นจึงมีเพียงรูปแบบการโพรบเท่านั้นที่เป็นปัจจัยซึ่งส่งผลต่อประสิทธิภาพของการดำเนินการต่าง ๆ ของตารางแฮช ผลการทดลองของเราแสดงให้เห็นว่าตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต (bucketed cuckoo hash table) ที่ใช้ฟังก์ชันแฮชสามฟังก์ชัน (BCHT) มีประสิทธิภาพเหนือกว่าวิธีทางเลือกอื่นที่ใช้ power-of-two choices, iceberg hashing และตารางแฮชคุกคูที่ใช้บักเก็ตขนาดหนึ่ง ที่ load factor สูงถึง 0.99 นั้น BCHT มีจำนวนการโพรบเฉลี่ยระหว่างการแทรก (insertion) เพียง 1.43 ครั้ง โดยใช้ฟังก์ชันแฮชเพียงสามฟังก์ชัน การค้นหาแบบพบ (positive query) และแบบไม่พบ (negative query) ต้องใช้จำนวนการโพรบเฉลี่ยต่อคีย์ไม่เกิน 1.39 และ 2.8 ครั้งตามลำดับ**

***คำสำคัญ*—GPU, ตาราง แฮช (hash tables), cuckoo hashing, power-of-two-choices, iceberg hashing**

I. บทนำ

ตาราง แฮช (hash table) เป็นโครงสร้างข้อมูลพื้นฐานที่ใช้ตอบคำถาม การเป็นสมาชิก (membership query) ("คีย์นี้มีอยู่หรือไม่") และการค้นหา (lookup) ("ค่าใดที่สัมพันธ์กับคีย์นี้") ระบบคอมพิวเตอร์ตั้งแต่ขนาดเล็กไปจนถึงขนาดใหญ่ล้วนพัฒนาตารางแฮชขึ้นโดยมีเป้าหมายการออกแบบและขีดความสามารถที่หลากหลาย การแลกเปลี่ยน (tradeoff) หลักในการออกแบบของการพัฒนาเหล่านี้คือการแลกเปลี่ยนระหว่างตัวชี้วัดสามประการ ได้แก่

ประสิทธิภาพการค้นหา (query performance) วัดเป็นจำนวนการค้นหาต่อวินาที และสะท้อนอัตราการประมวลผล (throughput) ของการค้นหาการเป็นสมาชิกหรือการค้นหาคู่คีย์–ค่า (key-value) ในตารางแฮช การค้นหาอาจเป็นแบบพบ (positive คือคีย์มีอยู่ในตารางแฮช) หรือแบบไม่พบ (negative คือคีย์ไม่มีอยู่ในตารางแฮช)

ประสิทธิภาพการสร้าง (build performance) วัด เป็น จำนวน คีย์หรือคู่คีย์–ค่าต่อวินาที และบ่งบอกอัตราการประมวลผลของการสร้างตารางแฮชจากรายการคีย์หรือคู่คีย์–ค่าที่ไม่ได้เรียงลำดับ

ประสิทธิภาพการใช้หน่วยความจำ (memory efficiency) แสดงว่าการพัฒนานั้นใช้หน่วยความจำที่จัดสรรไว้ได้ดีเพียงใด โดยเฉพาะ อย่างยิ่ง ตัวชี้วัด *load factor* คือ อัตราส่วน ระหว่างปริมาณหน่วยเก็บที่ใช้เก็บคีย์หรือคู่คีย์–ค่า กับปริมาณหน่วยเก็บทั้งหมดที่จัดสรรให้แก่ตารางแฮชทั้งตาราง

แม้ว่าปริภูมิการออกแบบ (design space) ของการพัฒนาตารางแฮชทั้งหมดจะกว้างขวางมาก แต่การมีตารางแฮชหลากหลายรูป

แบบที่เลือกการแลกเปลี่ยนแตกต่างกันก็เป็นประโยชน์ เนื่องจากแอปพลิเคชันต่าง ๆ อาจมีความต้องการด้านประสิทธิภาพที่แตกต่างกัน ในงานนี้ เรามุ่งเน้นที่ส่วนเฉพาะส่วนหนึ่งของปริภูมินี้ ได้แก่ ใช้ GPU (ของ NVIDIA) เพียงตัวเดียวเป็นแพลตฟอร์ม ชุดข้อมูลมีขนาดพอดีกับหน่วยความจำหลักของ GPU และเป็นตารางแฮชแบบสถิต (static hash table) ที่สร้างขึ้นเพียงครั้งเดียวบน GPU จากรายการคีย์หรือคู่คีย์–ค่า แล้วไม่มีการแก้ไขอีก ภายในส่วนนี้ เราพัฒนาและวิเคราะห์การออกแบบตารางแฮชแบบต่าง ๆ และเสนอแนะการออกแบบที่เหมาะสมสำหรับกรณีการใช้งานที่แตกต่างกัน

การออกแบบตารางแฮชที่เราพิจารณาได้รับอิทธิพลจากทางเลือกหลักสี่ประการ ได้แก่

รูปแบบการโพรบ (probing scheme) รูปแบบการโพรบเป็นเทคนิคต่าง ๆ สำหรับแก้ปัญหาการชนกัน (collision) ระหว่างคีย์หลายคีย์ที่แมปไปยังบักเก็ต (bucket) เดียวกันในตารางแฮช รูปแบบเหล่านี้ได้แก่ การโพรบเชิงเส้น (linear probing) การโพรบกำลังสอง (quadratic probing) การแฮชสองชั้น (double hashing) และ cuckoo hashing เมื่อเทียบกับ double hashing และ cuckoo hashing แล้ว รูปแบบการโพรบเชิงเส้นและการโพรบกำลังสองมีแนวโน้มที่จะเกิดการเกาะกลุ่ม (clustering) (เมื่อคีย์ที่ชนกันรวมตัวกันเป็นกลุ่ม)

ขนาดบักเก็ต (bucket size) กลยุทธ์ของเราวางรายการ (item) ลงในบักเก็ตของตารางแฮช โดยเราเป็นผู้เลือกขนาดของบักเก็ตเหล่านี้ การใช้บักเก็ตขนาดใหญ่ขึ้นช่วยลดความแปรปรวนของภาระ (load) ระหว่างบักเก็ต เมื่อขนาดบักเก็ตเพิ่มขึ้น load factor ที่ตารางแฮชทำได้ก็จะเพิ่มขึ้นด้วย การเลือกขนาดบักเก็ตได้รับอิทธิพลจากข้อจำกัดของฮาร์ดแวร์ ตัวอย่างเช่น เราอาจเลือกขนาดบักเก็ตให้เท่ากับขนาดของแคชไลน์ (cache line)

ความซับซ้อนของการโพรบ (probe complexity) ตารางแฮชสามารถวางคีย์ไว้ในบักเก็ตต่าง ๆ ได้ โดยแต่ละบักเก็ตเหล่านี้สัมพันธ์กับฟังก์ชันแฮช (hash function) ที่แตกต่างกัน การเพิ่มจำนวนฟังก์ชันแฮชช่วยปรับปรุง load factor ที่ทำได้ แต่ก็เพิ่มจำนวนบักเก็ตที่เราต้องตรวจสอบขณะแทรกหรือค้นหาคีย์ด้วย

กลยุทธ์การวาง (placement strategy) เนื่องจาก อาจ มี บัก เก็ต หลายบักเก็ตที่คีย์หนึ่ง ๆ อาจถูกวางลงไป ตารางแฮชที่มีฟังก์ชันแฮชหลายฟังก์ชันจึงต้องกำหนดกลยุทธ์การวาง ทางเลือกหนึ่งคือแทรกลงในบักเก็ตของฟังก์ชันแฮชแรกเสมอ ซึ่งหลีกเลี่ยงภาระส่วนเกิน (overhead) ของกลยุทธ์การวางทั้งหมด อีกทางเลือกหนึ่งคือวางคีย์ลงในบักเก็ตที่มีภาระน้อยที่สุดในบรรดาทางเลือกที่เป็นไปได้ ซึ่งเพิ่มภาระส่วนเกินจากการตรวจสอบบักเก็ตที่เป็นไปได้เหล่านี้

ในงานนี้ เรามุ่งเน้นที่ตารางแฮชคุกคู (cuckoo hash table) และสำรวจทางเลือกต่าง ๆ ข้างต้น เราพัฒนาและประเมินตารางแฮชที่ปรับให้เหมาะกับ GPU สามตระกูล ได้แก่

- ตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต (bucketed cuckoo hash table: BCHT): กลยุทธ์ cuckoo hashing ที่ขนาดบักเก็ตอาจเป็นหนึ่ง (ซึ่งตรงกับการพัฒนาของ CUDPP (1CHT)) หรือมากกว่าหนึ่ง
- ตารางแฮชแบบ power-of-two-choices ที่แบ่งเป็นบักเก็ต (bucketed power-of-two-choices hash table: BP2HT): คีย์จะเลือกระหว่างบักเก็ตที่เป็นไปได้สองบักเก็ต (หรือมากกว่า) ของตน โดยเลือกบักเก็ตที่มีภาระน้อยที่สุด
- ตารางแฮชไอซ์เบิร์ก (iceberg hash table: IHT): เราปรับให้เหมาะกับกรณีทั่วไปที่การแทรกสำเร็จโดยไม่มีการชนกัน โดยใช้ฟังก์ชันแฮชหลัก (primary hash function) เราพยายามเติมบักเก็ตจนถึงค่าขีดเริ่ม (threshold) จากนั้นเมื่อบักเก็ตถึงค่าขีดเริ่มแล้ว ฟังก์ชันแฮชเพิ่มเติมอีกสองฟังก์ชันจะให้ตำแหน่งทางเลือกสองตำแหน่งสำหรับการแทรก เราอธิบายกระบวนการนี้โดยละเอียดในหัวข้อที่ III-C

ผลงาน (contribution) ของเรามีดังนี้

- การออกแบบและพัฒนาตารางแฮชที่ปรับให้เหมาะกับ GPU ซึ่งมีการแลกเปลี่ยนและคุณสมบัติแตกต่างกัน
- การวิเคราะห์ประสิทธิภาพของแต่ละเทคนิค และการระบุปัจจัยที่จำกัดประสิทธิภาพ (performance limiter) ของแต่ละเทคนิค
- ข้อเสนอแนะการเลือกตารางแฮชสำหรับกรณีการใช้งานต่าง ๆ

ข้อเสนอแนะระดับสูง: เราพบว่าเมื่อข้อจำกัดหลักคือหน่วยความจำ และเราต้องการให้ตารางแฮชบรรลุ load factor สูง bucketed cuckoo hash ที่มีขนาดบักเก็ต 16 และใช้ฟังก์ชันแฮชสามฟังก์ชันให้ load factor สูงที่สุด (สูงถึง 0.98) นอกจากนี้ BCHT ยังให้อัตราการประมวลผลของการค้นหาและการแทรกสูงที่สุดด้วย เมื่อคุณสมบัติหลักที่ต้องการจากตารางแฮชคือความเสถียร (stability) ตารางแฮชไอซ์เบิร์กที่ใช้ขนาดบักเก็ต 16 (IHT) ให้อัตราการประมวลผลของการแทรกสูง แต่บรรลุ load factor ได้เพียง 0.82 การใช้ขนาดบักเก็ต 32 ทำให้ IHT บรรลุ load factor ได้สูงขึ้น (สูงถึง 0.91) แต่มีอัตราการประมวลผลของการแทรกและการค้นหาต่ำลง เมื่อต้องการทั้งความเสถียรและอัตราการประมวลผลของการค้นหาที่สูง BP2HT ที่มีขนาดบักเก็ต 16 เป็นทางเลือกที่ดีที่สุด แต่บรรลุ load factor ได้เพียงประมาณ 0.82 เช่นเดียวกับ IHT การเพิ่มขนาดบักเก็ตช่วยปรับปรุง load factor ที่ทำได้ (สูงถึง 0.91) โดยแลกกับอัตราการประมวลผลของทั้งการค้นหาและการแทรก ข้อเสนอแนะทั้งหมดข้างต้นรับประกันอัตราความสำเร็จ (success rate) ในการสร้างตารางอย่างน้อย 99% ตารางที่ I แสดงสรุปข้อเสนอแนะเหล่านี้

	Load factor	การแทรก	การค้นหา
การแทรก	BCHT, $b = 16$	—	—
การค้นหา	BCHT, $b = 16$	BCHT, $b = 16$	—
ความเสถียร	IHT, $b = 32$	IHT, $b = 16$	BP2HT, $b = 16$

ตารางที่ I: ข้อเสนอแนะการเลือกตารางแฮช แต่ละช่องในตารางแสดงตารางแฮชที่แนะนำสำหรับตัวชี้วัดของแถวและคอลัมน์นั้น ตัวอย่างเช่น หากต้องการบรรลุอัตราการประมวลผลของทั้งการแทรกและการค้นหาสูงที่สุด ตารางแฮชที่เราแนะนำคือ BCHT ที่มี $b = 16$

II. ความรู้พื้นฐาน: ตารางแฮชบน GPU และ Cuckoo Hashing

Cuckoo hashing [19] แตกต่างจากรูปแบบการแฮชแบบระบุที่อยู่เปิด (open addressing) ตรงที่รายการหนึ่ง ๆ สามารถแทรกลงได้เพียงหนึ่งในสองตำแหน่งที่เป็นไปได้เท่านั้น ทำให้การค้นหารวดเร็ว หากคีย์เก่าครอบครองทั้งสองตำแหน่งแล้ว ก็จะเกิดการชนกัน เราแก้ปัญหาการชนกันโดยดำเนินการ “cuckooing” ซึ่งคีย์ใหม่จะผลัก

คีย์เก่าคีย์หนึ่งออกไปยังตำแหน่งใหม่ Fotakis และคณะ [9] ได้ขยาย cuckoo hashing ให้เป็นกรณีทั่วไปและนำเสนอ d-ary cuckoo hashing ซึ่งจำนวนตำแหน่งที่เป็นไปได้ต่อคีย์คือ d (ในบทความนี้ เราใช้ h แทนจำนวนตำแหน่งที่เป็นไปได้) ด้วยการใช้ฟังก์ชันแฮชมากกว่าหนึ่งฟังก์ชัน พวกเขาบรรลุ load factor 0.97 และ 0.99 สำหรับฟังก์ชันแฮช 4 และ 5 ฟังก์ชันตามลำดับ Panigrahy [20] ได้ขยายขนาดบักเก็ต b ให้เป็นกรณีทั่วไปเพื่อให้บรรลุ load factor ที่สูงขึ้น โดยนำเสนอตารางแฮชคุกคูแบบ "แบ่งเป็นบักเก็ต" ("bucketized" cuckoo hash table: BCHT) ที่ใช้ $b = 2$ ต่อมา Erlingsson และคณะ [8] ได้รวมการขยายทั้งสองแบบเข้าด้วยกันและทดลองกับตารางแฮชคุกคูที่กำหนดพารามิเตอร์ด้วยทั้ง h และ b โดยทั่วไปปัญหา cuckoo hashing มักถูกมองเป็นปัญหาเชิงกราฟ ซึ่งบักเก็ตเป็นจุดยอด (vertex) และการแมปคีย์ไปยังบักเก็ตหนึ่งบักเก็ต (หรือมากกว่า) เป็นไฮเปอร์เอดจ์ (hyperedge) สำหรับทุกการผสมกันของขนาดบักเก็ตและจำนวนฟังก์ชันแฮช จะมีค่าขีดเริ่มของความหนาแน่นของไฮเปอร์เอดจ์ค่าหนึ่ง ซึ่งเมื่อต่ำกว่าค่าขีดเริ่มนี้ กราฟจะ*กำหนดทิศทางได้* (orientable) หรือกล่าวอีกนัยหนึ่งคือ เราสามารถสร้างตารางแฮชคุกคูได้ (ดู Walzer [21]) เมื่อจำนวนฟังก์ชันแฮชหรือขนาดบักเก็ตเพิ่มขึ้น ค่าขีดเริ่มก็จะเพิ่มขึ้น การบรรลุ load factor 100% เป็นไปได้เมื่อจำนวนฟังก์ชันแฮชและขนาดบักเก็ตเป็น (5, 5) และ (4, 6) การใช้ฟังก์ชันแฮชเพียงสองฟังก์ชันก็เพียงพอที่จะบรรลุ load factor มากกว่า 99% แต่ต้องใช้กลยุทธ์การกระจายภาระ (load-balancing) เมื่อแทรกลงในบักเก็ตใดบักเก็ตหนึ่งเหล่านี้

ตารางแฮชแรกที่สามารถสร้างบน GPU ได้ [1] ใช้การกำหนดรูปแบบตารางแฮชแบบ cuckoo hashing และต่อมามีการพัฒนา cuckoo hashing ที่เรียบง่ายกว่า [2] ซึ่งเก็บตารางแฮชทั้งหมดเป็นตารางเดียวในหน่วยความจำส่วนกลาง (global memory) การพัฒนานี้อยู่ในไลบรารี CUDPP [10] และนิยมใช้เป็นเกณฑ์เปรียบเทียบประสิทธิภาพในงานวิจัยด้าน GPU การพัฒนาของ CUDPP ใช้ $h = 4$ และ $b = 1$ งานวิจัยต่อมาได้สำรวจทั้งการแลกเปลี่ยนด้านประสิทธิภาพแบบต่าง ๆ และชุดคุณลักษณะของตารางแฮชที่กว้างขึ้น แทนที่จะสรุปงานเหล่านี้ไว้ที่นี่ เราขอแนะนำให้ผู้อ่านศึกษางานสำรวจ (survey) ฉบับสมบูรณ์ปี 2020 ของ Lessley และ Childs [14]

BCHT ของ Mega-KV ที่มีบักเก็ต 8 ช่อง (entry) และใช้ฟังก์ชันแฮชสองฟังก์ชัน มุ่งเป้าไปที่ที่เก็บคีย์–ค่าในหน่วยความจำ (in-memory key-value store) แบบผสม CPU–GPU [22] Mega-KV แบ่งตารางแฮชออกเป็นหลายพาร์ทิชัน (partition) และให้บล็อกเธรด (thread block) หนึ่งบล็อกประมวลผลแต่ละพาร์ทิชัน ความแตกต่างระหว่าง BCHT ของเราที่มี $b = 8$ กับ Mega-KV คือ เราใช้ฟังก์ชันแฮชสามฟังก์ชันแทนที่จะใช้เพียงสองฟังก์ชัน และใช้ตารางแฮชเพียงตารางเดียวแทนที่จะใช้หลายพาร์ทิชัน การใช้ฟังก์ชันแฮชสามฟังก์ชันทำให้ BCHT ของเราบรรลุ load factor และอัตราการประมวลผลของการแทรกที่สูงขึ้น อย่างไรก็ตาม ข้อเสียของการใช้ฟังก์ชันแฮชเพิ่มอีกหนึ่งฟังก์ชันคือ อัตราการประมวลผลของการค้นหาจะลดลง (1.38 เท่าที่ load factor 0.96) เมื่อการค้นหาทั้งหมดเป็นแบบไม่พบ ดังที่เราจะเห็นในผลการทดลองของเรา (หัวข้อที่ VII) การใช้บักเก็ตขนาด 16 ให้ประสิทธิภาพเหนือกว่าขนาดบักเก็ตทางเลือกอื่น ๆ

ตาราง Horton (Horton tables) [6] ปรับให้เหมาะกับการค้นหาแบบไม่พบในตารางแฮช โดยเพิ่ม "ตารางแมปใหม่" ("remap tables") เข้าไปใน BCHT เพื่อลดจำนวนการเข้าถึงหน่วยความจำที่คาดหมายของ BCHT ตาราง Horton แฮชคีย์ไปยังบักเก็ตหลักโดยใช้ฟังก์ชันแฮชเพียงฟังก์ชันเดียว ส่วนบักเก็ตรองจะถูกเลือกจากฟังก์ชันแฮชรอง 7 ฟังก์ชัน โดยเลือกบักเก็ตที่มีภาระน้อยที่สุด พวกเขาบรรลุ load factor 95% และจำนวนการโพรบเฉลี่ย 1.18 และ 1.06 สำหรับการค้นหาแบบพบและแบบไม่พบตามลำดับ อย่างไรก็ตาม การสร้างตารางดำเนินการบน CPU งานของเรามุ่งเน้นการดำเนินการทั้งการสร้างและการค้นหาบน GPU ทั้ง Mega-KV และ Horton ไม่ได้เปรียบเทียบประสิทธิภาพกับตารางแฮชคุกคูแบบดั้งเดิมที่มี $b = 1$ บน GPU แต่ทั้งสองได้ผลลัพธ์ที่เร็วกว่า CUDPP ทั้งสำหรับการค้นหาแบบพบและแบบไม่พบ

WarpDrive [13] และงานต่อยอดใน WarpCore [12] ได้นำเสนอตารางแฮชแบบสถิตชนิดระบุที่อยู่เปิดที่ใช้รูปแบบ double hashing โดยใช้ cooperative groups ของ CUDA[1] พวกเขาทดลองกับขนาดหน้าต่างการโพรบ (probe window) ต่าง ๆ ประสิทธิภาพที่ดีที่สุดเกิดขึ้นเมื่อขนาดหน้าต่าง $|g| \in \{2, 4, 8\}$ พวกเขาได้ประสิทธิภาพการแทรกและการค้นหาเร็วกว่า CUDPP สูงสุด 2.84 เท่าและ 1.3 เท่า ความแตกต่างหลักระหว่างแนวทางของเรากับของ WarpCore คือการเลือกรูปแบบการโพรบ เมื่อเทียบกับ WarpCore แล้ว ตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ตของเรา (BCHT ที่มี $b = 16$) มีประสิทธิภาพการแทรกใกล้เคียงกัน และมีประสิทธิภาพการค้นหาแบบพบเร็วกว่าเล็กน้อย ($\leq$ 1.2 เท่า) สำหรับการค้นหาแบบไม่พบ BCHT ให้อัตราเร่ง (speedup) {2.3 เท่า, 1.6 เท่า} เมื่อเทียบกับ WarpCore ที่มีหน้าต่างการโพรบ {4, 8} ประสิทธิภาพการค้นหาแบบไม่พบของ BCHT เหนือกว่า WarpCore อย่างมีนัยสำคัญที่ load factor สูง โดยมีอัตราเร่งสูงถึง 6.4 เท่า[2] แม้ว่าประสิทธิภาพอาจใกล้เคียงกันสำหรับการดำเนินการบางอย่าง แต่ประสิทธิภาพที่ต่ำจนไม่อาจยอมรับได้ของ WarpCore สำหรับการค้นหาแบบไม่พบเป็นเหตุผลที่เราแนะนำ BCHT

โดยทั่วไป การพัฒนา cuckoo hashing บน GPU ไม่ได้มุ่งเป้าไปที่สถานการณ์แบบพลวัต (dynamic) ซึ่งมีการแทรกรายการใหม่ลงในหรือลบรายการออกจากตารางแฮชที่มีอยู่แล้ว อย่างไรก็ตาม Li และคณะ [15] ได้เสนอการพัฒนาตารางแฮชคุกคูแบบพลวัต (DyCuckoo) เมื่อไม่นานมานี้ เพื่อหลีกเลี่ยงการสร้างตารางแฮชทั้งหมดใหม่ตั้งแต่ต้น พวกเขาแบ่งตารางออกเป็นตารางย่อย (subtable) หลายตาราง แต่ละตารางย่อยมีขอบเขตล่างและขอบเขตบนของ load factor และเมื่อจำเป็นต้องปรับขนาด พวกเขาจะเพิ่มขนาดของตารางย่อยที่เล็กที่สุดเป็นสองเท่า (หรือลดขนาดตารางย่อยที่เล็กที่สุดลงครึ่งหนึ่ง) คีย์หนึ่ง ๆ สามารถแฮชไปยังตารางย่อยได้เพียงสองตารางเท่านั้น ดังนั้นการค้นหาจึงต้องค้นดู (lookup) ไม่เกินสองครั้ง

ในสถานการณ์แบบสถิต DyCuckoo มีประสิทธิภาพใกล้เคียงกับ Mega-KV และ WarpDrive เมื่อเทียบกับ BCHT ของเราที่มี $b = 16$ การแทรกของเราเร็วกว่า DyCuckoo 1.1 เท่า การค้นหาแบบพบใน BCHT ได้อัตราเร่งสูงกว่า โดยมีอัตราเร่งเฉลี่ย[2] 1.2 เท่า สำหรับการค้นหาแบบไม่พบ BCHT ได้อัตราเร่งอย่างมีนัยสำคัญ {2.1 เท่า, 1.1 เท่า} ที่ load factor ระหว่าง {0.6, 0.93} อย่างไรก็ตาม ที่ load factor สูงกว่านั้น DyCuckoo เริ่มมีประสิทธิภาพเหนือกว่า BCHT

[1] https://developer.nvidia.com/blog/cooperative-groups/

[2] ผลลัพธ์เฉลี่ยจาก load factor ต่าง ๆ สำหรับการแทรกและการค้นหาคีย์ที่ไม่ซ้ำกัน 50 ล้านคีย์

ของเรา โดยมีอัตราเร่ง 1.1 เท่าที่ load factor 0.97 DyCuckoo มีประสิทธิภาพเหนือกว่า BCHT ของเรา เนื่องจาก BCHT ดำเนินการโพรบสูงสุดสามครั้งเมื่อคีย์ไม่มีอยู่ในตารางแฮช ในขณะที่ DyCuckoo ต้องค้นดูเพียงสองครั้งในกรณีเลวร้ายที่สุด เราไม่ได้กล่าวถึงการปรับปรุงแบบพลวัต (dynamic update) ในงานนี้

A. *กลยุทธ์ระดับสูงในการพัฒนา Cuckoo Hashing บน GPU*

ในการพัฒนาของ CUDPP เช่นเดียวกับงานส่วนใหญ่ที่ตามมา การค้นหาจะถูกประมวลผลแบบขนานข้ามเธรด CUDA (คีย์ที่ค้นหาแต่ละคีย์ถูกมอบหมายให้เธรดที่ต่างกัน) แต่การคำนวณของการค้นหาแต่ละครั้งเป็นแบบลำดับ (serial) ภายในเธรด เธรดจะตรวจสอบแต่ละบักเก็ต $H_i(k)$ และคืนค่า "พบ" หากคีย์ถูกเก็บอยู่ในบักเก็ตใดบักเก็ตหนึ่งเหล่านั้น และคืนค่า "ไม่พบ" ในกรณีอื่น เนื่องจากการค้นหาแต่ละครั้งทำงานแบบลำดับภายในเธรด เราจึงสามารถหยุดการค้นหาได้ (โดยไม่ต้องเข้าถึงหน่วยความจำเพิ่มเติม) เมื่อพบรายการ

เช่นเดียวกับการค้นหา การแทรกถูกประมวลผลแบบขนานข้ามเธรด แต่กระบวนการแทรกแต่ละครั้ง (ซึ่งอาจเกี่ยวข้องกับการสลับหลายครั้ง) เป็นแบบลำดับภายในเธรด กล่าวอีกนัยหนึ่งคือ กระบวนการแทรกเริ่มต้นด้วยรายการที่จะวาง และอาจมีการสลับ (exchange) หลายครั้ง (cuckooing) และสิ้นสุดเมื่อรายการหนึ่งถูกวางลงในช่องว่าง (empty slot) ได้ในที่สุด การดำเนินการสลับแต่ละครั้งต้องเป็นแบบอะตอมิก (atomic) เพื่อรับประกันการทำงานที่ถูกต้อง ดังนั้นการแทรกลงในบักเก็ตเดียวกันจะถูกทำให้เป็นลำดับ แต่ความสามารถในการแทรกลงในบักเก็ตต่าง ๆ พร้อมกันทำให้เกิดภาวะพร้อมกัน (concurrency) ได้มาก เนื่องจากการสลับเป็นแบบอะตอมิก ขนาดที่ใช้ได้จริงของคีย์หรือคู่คีย์–ค่าจึงถูกจำกัดด้วยขนาดของการสลับแบบอะตอมิก (atomic exchange) ที่ฮาร์ดแวร์รองรับ ไลบรารีมาตรฐาน CUDA C++ ใหม่ของ NVIDIA คือ `libcu++`[3] ซึ่งออกรุ่นแรกในเดือนตุลาคม 2020 สัญญาว่าจะรองรับการดำเนินการอะตอมิกแบบทั่วไปกับชนิดข้อมูลใด ๆ แต่ยังไม่ทราบคุณลักษณะด้านประสิทธิภาพสำหรับชนิดข้อมูลใด ๆ ก็ตาม ไม่น่าจะมีประสิทธิภาพเทียบเท่ากับการสลับแบบอะตอมิกในปัจจุบัน

การพัฒนาอาจเลือกค้นหาบักเก็ตแบบลำดับหรือแบบขนาน[4] และอาจเลือกกลยุทธ์การแทรกลงบักเก็ตและการค้นหาบักเก็ตที่ร่วมกันลดจำนวนการเข้าถึงหน่วยความจำที่คาดหมายให้น้อยที่สุด ตัวอย่างเช่น หากเราเลือกฟังก์ชันแฮชสำหรับการแทรกตามลำดับ $H_0, H_1, \ldots$ เสมอ และต่อมาค้นหาแบบลำดับโดยใช้ลำดับเดียวกัน เราก็สามารถหยุดการค้นหาได้หากพบช่องว่าง เนื่องจากเราไม่รองรับการลบ

คุณลักษณะด้านประสิทธิภาพของ cuckoo hashing บน GPU: ในกรณีที่ดีที่สุด (และเป็นกรณีทั่วไป) สำหรับการค้นหา คีย์จะถูกพบในบักเก็ตแรกที่ค้นหา หมายความว่าการเข้าถึงหน่วยความจำเพียงอย่างเดียวที่จำเป็นในการค้นหาคีย์คือการอ่านแบบสุ่ม (random read) เพียงครั้งเดียวจากหน่วยความจำส่วนกลางที่เก็บตารางแฮชไว้ สำหรับการแทรก กรณีอุดมคติคือเมื่อตำแหน่งแรกที่ตรวจสอบว่างอยู่ จึงไม่จำเป็นต้องทำ cuckooing ในกรณีนี้ การเข้าถึงหน่วยความจำเพียงอย่างเดียวที่จำเป็นในการแทรกคีย์ลงในตารางคือการสลับแบบอะตอมิกหนึ่งครั้งในหน่วยความจำส่วนกลาง อย่างไรก็ตาม เมื่อตารางเริ่มเต็ม จำนวนการสลับเฉลี่ยจะเพิ่มขึ้นอย่างรวดเร็ว นอกจากนี้ การสลับเหล่านี้ยังเพิ่มต้นทุนของการค้นหา เนื่องจากย้ายรายการออกจากช่องหลัก (primary slot) ไปยังช่องรอง (secondary slot) ต้นทุนของกรณีอุดมคติสำหรับการค้นหาและการแทรกนั้นต่ำ หรืออาจกล่าวได้ว่าน้อยที่สุด หากกรณีทั่วไปใกล้เคียงกับกรณีอุดมคติ เมื่อมองในแง่อัตราการประมวลผลบน GPU แล้ว การพัฒนา cuckoo hashing ก็ยากที่จะมีวิธีใดเอาชนะได้ ในบทความปี 2011 ของ Alcantara และคณะ ได้ระบุว่าประสิทธิภาพของหน่วยความจำเป็นคอขวด (bottleneck) [2] ว่า "… we can run up to 1000 instructions per memory update and remain memory bound. Consequently, arithmetic utilization and thread divergence have only marginal impact on hash performance." ("… เราสามารถรันคำสั่งได้สูงสุด 1000 คำสั่งต่อการปรับปรุงหน่วยความจำหนึ่งครั้งโดยยังคงถูกจำกัดด้วยหน่วยความจำ ดังนั้นการใช้งานหน่วยคำนวณเลขคณิตและการแตกแขนงของเธรด (thread divergence) จึงมีผลกระทบต่อประสิทธิภาพการแฮชเพียงเล็กน้อยเท่านั้น") เนื่องจาก GPU รุ่นใหม่มีปริมาณการคำนวณต่อการดำเนินการหน่วยความจำเพิ่มขึ้นเรื่อย ๆ ("ความเข้มข้นของการคำนวณ" (compute intensity) สูงขึ้น) เราจึงคาดว่าข้อสรุปนี้จะยิ่งเป็นจริงมากขึ้นกับสถาปัตยกรรมในอนาคต ทั้งสำหรับการค้นหาและการแทรก

สมมติฐานข้างต้นเกี่ยวกับต้นทุนหลักของการแทรก (การสลับแบบอะตอมิก) หรือการค้นหา (การอ่านจากหน่วยความจำส่วนกลาง) ตั้งอยู่บนสมมติว่าการแปลงที่อยู่ระหว่างหน่วยความจำเสมือน (virtual memory) และหน่วยความจำกายภาพ (physical memory) ไม่มีต้นทุน ซึ่งไม่ถูกต้องเมื่อเกิด TLB miss ตัวอย่างเช่น บนสถาปัตยกรรมไมโคร (microarchitecture) Volta ของ NVIDIA นั้น Jia และคณะ [11] ชี้ว่า L2 TLB ของ Volta เก็บรายการได้เพียงพอที่จะครอบคลุมหน่วยความจำ 8 GB เท่านั้น ดังนั้นเราจึงคาดว่าประสิทธิภาพจะลดลงเนื่องจาก TLB miss เมื่อตารางแฮชคุกคูมีขนาดใหญ่กว่านี้ เราไม่ได้กล่าวถึงประเด็นนี้ในงานนี้

III. อัลกอริทึม

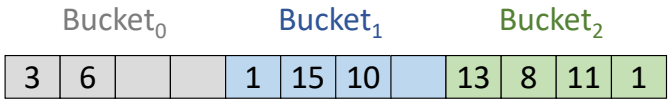
ในงานนี้ เราอธิบายการพัฒนาตารางแฮชสามแนวทางที่แตกต่างกัน ได้แก่ ตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต (BCHT) ตารางแฮชแบบ power-of-two-choices ที่แบ่งเป็นบักเก็ต (BP2HT) และตารางแฮชไอซ์เบิร์ก (IHT) BCHT เป็นรูปแบบทั่วไปของการพัฒนาตารางแฮชคุกคูของ CUDPP (1CHT) โดยขนาดบักเก็ตของ BCHT สามารถมากกว่าหนึ่งได้ ในหัวข้อนี้ เราอภิปรายอัลกอริทึมการแทรกและการค้นหาของแต่ละแนวทาง ตารางแฮชแทนด้วยอาร์เรย์ขนาดคงที่ที่แบ่งออกเป็นบักเก็ตจำนวน m บักเก็ต แต่ละบักเก็ตมีขนาด b (กล่าวคือบักเก็ตหนึ่งมี b ตำแหน่งสำหรับ b คู่) อาร์เรย์นี้มีความจุ (capacity) C คู่คีย์–ค่า โดย $C = m \times b$ ตารางแฮชที่บรรจุ n คีย์มี load factor เท่ากับ n/C

A. *ตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต (BCHT หรือ 1CHT)*

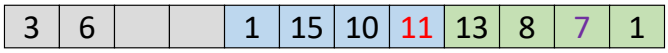
เราเริ่มต้นด้วยการกำหนดฟังก์ชันแฮช h ฟังก์ชัน $H_0, H_1 \ldots H_{h-1}$ จากนั้นจัดสรรตารางที่มี m บักเก็ต โดยแต่ละบักเก็ตมีตำแหน่งที่ใช้ได้ b ตำแหน่ง เพื่อความเรียบง่าย เราจะ

[3] https://nvidia.github.io/libcudacxx/

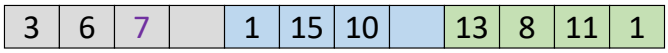
[4] การค้นหาแบบขนานมีเวลาแฝง (latency) โดยรวมต่ำกว่าการค้นหาแบบลำดับ แต่ก็อาจเพิ่มจำนวนการเข้าถึงหน่วยความจำด้วย



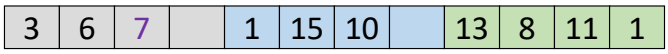
(a) ตารางแฮชเริ่มต้นที่มีสามบักเก็ต และ $b = 4$



(b) ใน BCHT คีย์ถูกแฮชไปยังบักเก็ตที่เต็มแล้ว $h_0(7) = 2$ เราสลับคีย์ใหม่กับคีย์หนึ่งที่สุ่มเลือกจากบักเก็ตที่เต็ม คีย์ที่ถูกสลับออกจะถูกย้ายไปยังบักเก็ตถัดไปตามฟังก์ชันแฮชสามฟังก์ชัน $h_0(11) = 0$, $h_1(11) = 2$ และ $h_2(11) = 1$



(c) ใน BP2HT เราคำนวณฟังก์ชันแฮชสองฟังก์ชัน $h_0(7) = 0$ และ $h_1(7) = 1$ จากนั้นแทรกคีย์ลงในบักเก็ตที่มีภาระน้อยที่สุด



(d) ใน IHT ที่มีค่าขีดเริ่ม $t = 3$ คีย์จะถูกแมปไปยังบักเก็ตหลักของตนก่อน $h_p(7) = 1$ เนื่องจากบักเก็ตเกินค่าขีดเริ่ม เราจึงคำนวณฟังก์ชันแฮชรองสองฟังก์ชัน: $h_{s0}(2) = 0$ และ $h_{s1}(2) = 0$ คีย์ใหม่จะถูกแทรกลงในบักเก็ตที่มีภาระต่ำกว่า

รูปที่ 1: การแทรกคีย์ $= 7$ ลงในตารางแฮชแบบต่าง ๆ

กล่าวถึงเฉพาะการเก็บคีย์ (โดยทั่วไปค่าจะถูกจัดการโดยรวมเข้ากับคีย์ที่สัมพันธ์กัน)

ฟังก์ชัน แฮช จำนวน h ฟังก์ชัน แมป คีย์ k ไป ยัง บัก เก็ต $H_0(k) \dots H_{h-1}(k)$ ดังนั้นกระบวนการค้นหาจึงต้องตรวจสอบเพียงบักเก็ตเหล่านี้ คีย์จะอยู่ในตารางแฮชก็ต่อเมื่อพบคีย์ในบักเก็ตใดบักเก็ตหนึ่งเหล่านี้เท่านั้น เราใช้ฟังก์ชันแฮชจำนวนคงที่ ดังนั้นการค้นหาจึงมีความซับซ้อน $O(1)$

การสร้างตารางมีความซับซ้อนกว่า (เล็กน้อย) และแสดงให้เห็นถึงแนวคิดเฉพาะตัวของ cuckoo hashing เรากำหนดค่าเริ่มต้นให้ทุกบักเก็ตของตารางแฮชว่างเปล่า จากนั้นแทรกแต่ละคีย์ลงในตารางแฮช สำหรับคีย์ k เราเริ่มต้นด้วยฟังก์ชันแฮชแรก H_0 และพยายามแทรก k ลงในบักเก็ต $H_0(k)$ หากบักเก็ตว่าง เราแทรกคีย์และเสร็จสิ้น หากบักเก็ตเต็ม เราจะ*สลับ* (exchange) รายการ k ของเรากับ $\hat{k}$ ซึ่งเป็นคีย์สุ่มที่อยู่ใน $H_0(k)$ อยู่แล้ว การสลับนี้เองคือลักษณะเฉพาะของ cuckoo hashing และเรากล่าวว่า k *cuckoo* $\hat{k}$ ขณะนี้คีย์ k ถูกเก็บไว้ในตารางแฮชที่ $H_0(k)$ และเราต้องแทรก $\hat{k}$ ต่อ หาก $\hat{k}$ เคยถูกเก็บโดยใช้ H_i (โดยทั่วไป) เราจะเริ่มการแทรกครั้งใหม่นี้ด้วยฟังก์ชันแฮชลำดับถัดไป คือ $H_{(i+1) \bmod h}$[5] เราเสร็จสิ้นเมื่อเก็บคีย์ของเราลงในบักเก็ตว่างได้สำเร็จ หรือเมื่อถึงจำนวนการสลับสูงสุด ซึ่งในกรณีหลังนี้ถือว่าการสร้างตารางแฮชล้มเหลว การวิเคราะห์เชิงทฤษฎีของ cuckoo hashing ต้องการ load factor ค่อนข้างต่ำเพื่อรับประกันว่าการสร้างจะล้มเหลวด้วยความน่าจะเป็นต่ำ แต่ในทางปฏิบัติ cuckoo hashing สามารถบรรลุ load factor สูงได้ ข้อดีของการเลือกฟังก์ชันแฮชตามลำดับที่เพิ่มขึ้นคือ ระหว่างการค้นหา หากบักเก็ตใดมีคีย์ว่าง การค้นหาสามารถออกก่อนกำหนด (exit early) ได้

ตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ตอีกรูปแบบหนึ่งใช้รูปแบบการกระจายภาระเมื่อเลือกบักเก็ตจาก h บักเก็ตที่เป็นไปได้ แทนที่จะแทรกคีย์ลงในบักเก็ตแรกเสมอดังที่กล่าวไปก่อนหน้านี้ เราแทรกคีย์ลงในบักเก็ตที่มีภาระน้อยที่สุดในบรรดาบักเก็ตที่เป็นไปได้ทั้งหมด เทคนิคการกระจายภาระช่วยปรับปรุง load factor ที่ทำได้ โดยแลกกับการถ่ายโอนหน่วยความจำเพิ่มเติมที่จำเป็นสำหรับการประเมินภาระของบักเก็ตที่เป็นไปได้ทั้ง h บักเก็ต เนื่องจากการแทรกอาจเลือกบักเก็ตใดก็ได้จากบักเก็ตที่เป็นไปได้ ระหว่างการค้นหาจึงไม่สามารถใช้กลยุทธ์การออกก่อนกำหนดได้ ส่งผลให้เกิดภาระส่วนเกินคล้ายกับภาระส่วนเกินของการแทรก (เว้นแต่จะพบคีย์ในบักเก็ตแรก ๆ) ต่อไปเราจะอภิปรายการกระจายภาระ (power of two) ในฐานะเทคนิคในตัวเอง

B. พลังของสองทางเลือก (หรือมากกว่า) (P2HT)

พิจารณาตารางแฮชคุกคูที่มีฟังก์ชันแฮชสองฟังก์ชัน เมื่อเราแทรกคีย์ใหม่ เราประเมินฟังก์ชันแฮชหนึ่งฟังก์ชันและแทรกคีย์นั้นลงในบักเก็ตที่สอดคล้องกัน ฟังก์ชันแฮชอีกฟังก์ชันหนึ่งจะถูกใช้ก็ต่อเมื่อคีย์ของเราถูกผลักออก (eject) และแทรกใหม่เท่านั้น

แต่ถ้าเราประเมิน*ทั้งสอง*ฟังก์ชันแฮชก่อนตัดสินใจว่าจะแทรกที่ใดเล่า? เมื่อมีข้อมูลนี้ เราสามารถและต้องการที่จะเลือกบักเก็ตที่มีภาระน้อยกว่า เราคาดการณ์ว่าเราจะบรรลุ load factor ที่สูงขึ้นได้ด้วยกลยุทธ์นี้ โดยแลกกับการเข้าถึงหน่วยความจำที่มากขึ้น (เนื่องจากตอนนี้เราต้องตรวจสอบหลายบักเก็ตในการแทรกแต่ละครั้ง และเราไม่มีบักเก็ตที่ต้องการเป็นพิเศษสำหรับการค้นหาอีกต่อไป)

เทคนิคนี้ซึ่งเรียกว่า "พลังของสองทางเลือก" ("power of two choices") เป็นเทคนิคที่ทรงพลัง: "even a small amount of choice can lead to drastically different results in load balancing" ("แม้ทางเลือกเพียงเล็กน้อยก็สามารถนำไปสู่ผลลัพธ์ที่แตกต่างกันอย่างมากในการกระจายภาระ") [17] เมื่อมี h ทางเลือก ภาระสูงสุดสำหรับ n คีย์จะลดลงจาก $O(\log n / \log \log n)$ เป็น $O(\log \log n / \log h)$ [4]

ต่างจาก cuckoo hashing ตรงที่ power-of-two-choices hashing มี*ความเสถียร* (stable): เมื่อเราวางคีย์แล้ว คีย์นั้นจะไม่ถูกย้ายอีกเลย ความเสถียรมีประโยชน์ในแอปพลิเคชันที่ต้องการพอยน์เตอร์ (pointer) (ที่เสถียร) ไปยังคีย์หนึ่ง ๆ ในตารางแฮช เช่นเดียวกับตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต เราสามารถสร้าง power-of-two-choices แบบแบ่งเป็นบักเก็ต (BP2HT) ที่ขนาดบักเก็ตมากกว่าหนึ่งได้

C. Iceberg Hashing

ในตารางแฮชที่มีความเสถียร บักเก็ตที่ใหญ่ขึ้นช่วยเพิ่มอัตราการประมวลผลของการค้นหาและการสร้าง แต่ไม่ทำให้เราบรรลุ load factor สูงได้ Power-of-two-choices hashing สามารถปรับปรุง load factor ได้ แต่ก็เพิ่มปริมาณงานที่เราต้องทำสำหรับการค้นหาและการแทรก แม้เราจะสามารถรวมสองเทคนิคนี้เข้าด้วยกันโดยตรงได้ แต่การทำเช่นนั้นไม่ได้ให้ทั้งประสิทธิภาพที่เพิ่มขึ้นและประสิทธิภาพการใช้หน่วยความจำที่เพิ่มขึ้นตามที่เราตั้งเป้าไว้

สิ่งที่เราชอบเกี่ยวกับบักเก็ตขนาดใหญ่คือพฤติกรรมในกรณีทั่วไป: เรามีแนวโน้มสูงที่จะพบคีย์ที่กำลังค้นหา หรือแทรกได้สำเร็จในบัก

[5] เราสามารถระบุได้ว่าฟังก์ชันแฮชใดใน h ฟังก์ชันของเราที่ใช้เก็บคีย์หนึ่ง ๆ ไว้ที่ที่อยู่หนึ่ง ๆ $H_i(k)$ โดยแฮชคีย์ของเราด้วยฟังก์ชันแฮชทั้ง h ฟังก์ชัน แล้วดูว่าฟังก์ชันแฮชใดให้ที่อยู่ในตารางแฮช $H_i(k)$ โดยตัดสินกรณีเสมอกันด้วยการเลือกฟังก์ชันแฮชที่มีดัชนีต่ำที่สุด

เก็ตแรก สิ่งที่เราชอบเกี่ยวกับ power-of-two-choices คือการให้การใช้หน่วยความจำที่ดีกว่าสำหรับกรณีพิเศษ (extraordinary case) สำหรับกรณีเหล่านี้ เรายินดีจ่ายต้นทุนเพิ่มเติมของการมีหลายทางเลือก

Iceberg hashing [18] รวมพฤติกรรมในกรณีทั่วไปและกรณีพิเศษเข้าด้วยกัน[6] ในรูปแบบใหม่ ก่อนอื่นเราจะสร้างความเข้าใจเชิงสัญชาตญาณ พิจารณาการเติมตารางแฮชส่วนใหญ่ด้วยต้นทุนต่ำโดยวางคีย์ไว้ในบักเก็ตแรกที่คีย์ใส่ลงได้ จากนั้นเติมส่วนที่เหลือของตารางโดยใช้แนวทางที่ปรับการวางให้เหมาะสม (กล่าวคือ power-of-two-choices) คำว่า "iceberg hashing" สะท้อนกลยุทธ์สองแบบที่แตกต่างกันนี้: กรณีทั่วไป (ซึ่งพบมากที่สุด เปรียบได้กับส่วนที่ใหญ่ที่สุดของภูเขาน้ำแข็งที่อยู่ใต้น้ำ) และกรณีพิเศษ (ส่วนเล็ก ๆ ของภูเขาน้ำแข็งที่อยู่เหนือน้ำ)

เช่นเดียวกับ power-of-two-choices hashing นั้น iceberg hashing มีความเสถียร

1) อัลกอริทึม iceberg hashing:

a) ข้อกำหนดเบื้องต้น: เลือกค่าขีดเริ่ม t สำหรับบักเก็ต t เป็นเส้นแบ่งระหว่างกรณี "ทั่วไป" ที่กล่าวถึงข้างต้น (บักเก็ตมีคีย์น้อยกว่า t คีย์) กับกรณี "พิเศษ" (มีอย่างน้อย t คีย์) หากแต่ละบักเก็ตมีคีย์โดยเฉลี่ย b_{avg} คีย์ ให้กำหนด t ให้มากกว่า b_{avg} เล็กน้อย

เลือกฟังก์ชันแฮช 3 ฟังก์ชัน (H_p, H_{s0}, H_{s1}) H_p คือฟังก์ชันแฮชสำหรับบักเก็ต "หลัก" (primary) ส่วน H_s ทั้งสองฟังก์ชันใช้สำหรับบักเก็ต "รอง" (secondary)

b) การแทรก: ในการแทรกคีย์ k:

- คำนวณฟังก์ชันแฮชหลัก $H_p(k)$ ของคีย์ หากบักเก็ตผลลัพธ์มีคีย์น้อยกว่า t คีย์ ให้แทรกรายการลงที่นั่นแล้วคืนค่า
- หากบักเก็ตผลลัพธ์เต็มเกินไป ให้คำนวณฟังก์ชันแฮชรอง*ทั้งสอง*ฟังก์ชัน $H_{s0}(k)$ และ $H_{s1}(k)$ แล้วแทรกคีย์ลงในบักเก็ตที่เต็มน้อยกว่าในสองบักเก็ตนั้น
- หากบักเก็ตทั้งหมดเหล่านี้เต็ม การสร้างของเราถือว่าล้มเหลว ให้เลือกชุดฟังก์ชันแฮชชุดอื่นหรือขยายตาราง แล้วลองใหม่ เราคาดว่าสถานการณ์นี้จะเกิดขึ้นด้วยความน่าจะเป็นต่ำมากเท่านั้นหากเลือกค่าขีดเริ่มและขนาดบักเก็ตอย่างเหมาะสม

c) การค้นหา: ในการค้นหาคีย์ k:

- ตรวจสอบตำแหน่ง $H_p(k)$ ก่อน คืนค่าคีย์หากพบ
- หากไม่พบคีย์ใน $H_p(k)$ ให้ตรวจสอบทั้งสองตำแหน่ง $H_{s0}(k)$ และ $H_{s1}(k)$ คืนค่ารายการหากพบ
- หากไม่พบคีย์ในตำแหน่งใดเลย แสดงว่าคีย์ไม่อยู่ในตารางแฮช

IV. การตัดสินใจเชิงออกแบบร่วม

เมื่อพัฒนากลยุทธ์ตารางแฮชแบบต่าง ๆ ข้างต้น เราพบการตัดสินใจเชิงออกแบบร่วมกันบางประการ ได้แก่ เราจะจัดสรรรายการกี่รายการต่อบักเก็ต เราจะใช้ฟังก์ชันแฮชกี่ฟังก์ชันในตารางแฮช และเราจะตัดสินใจวางรายการอย่างไรเมื่อมีหลายทางเลือก ต่อไปเราจะอภิปรายการตัดสินใจเชิงออกแบบเหล่านี้และสัญชาตญาณเบื้องหลัง

[6]การแยกงานคำนวณออกเป็นส่วนกรณีทั่วไปและส่วนกรณีพิเศษเคยถูกสำรวจมาก่อนแล้วในโดเมนแอปพลิเคชัน GPU อื่น ๆ เช่น การคูณเมทริกซ์แบบเบาบาง (sparse matrix) กับเวกเตอร์แบบหนาแน่น (dense vector) [5]

A. ขนาดบักเก็ต

ผลของขนาดบักเก็ตต่อตารางแฮชคุกคู: พิจารณาตารางแฮชที่ใช้ฟังก์ชันแฮชสองฟังก์ชัน $h = 2$ และมีขนาดบักเก็ตหนึ่ง $b = 1$ ในกรณีนี้ รายการหนึ่ง ๆ จะอยู่ในตำแหน่งแรกด้วยความน่าจะเป็น $1/2$ ทีนี้ลองพิจารณาบักเก็ตที่ใหญ่ขึ้นคือ $b = 16$ ในกรณีนี้ ภายใต้สมมติฐานที่ทำให้ง่ายขึ้นบางประการ เราเพิ่มความน่าจะเป็นที่จะพบรายการในตำแหน่งแรกเป็น $1 - 1/2^{16}$ ดังนั้น ด้วยพารามิเตอร์เหล่านี้ เราจะเพิ่มเวลาในการคำนวณประมาณ $10\times$ (ในกรณี $b = 1$ เราจะทำการเปรียบเทียบโดยคาดหมาย 1.5 ครั้งต่อการค้นหา ในขณะที่ในกรณีสมมติ $b = 16$ เราจะทำการเปรียบเทียบมากกว่า 16 ครั้งเล็กน้อย) การคำนวณเหล่านี้เป็นเพียงการประมาณคร่าว ๆ แต่ก็แสดงให้เห็นว่าบักเก็ตที่ใหญ่ขึ้นสามารถลดการใช้หน่วยความจำได้โดยแลกกับการคำนวณที่เพิ่มขึ้น ซึ่งควรส่งผลให้อัตราการประมวลผลของระบบโดยรวมดีขึ้น

การเพิ่มขนาดบักเก็ตยังสามารถลดปริมาณ cuckooing ที่คาดหมายได้ด้วย เพื่อให้เห็นเช่นนี้ สังเกตว่าหาก $b = 1$ จะมีรายการเพียงรายการเดียวเท่านั้นที่แทรกลงในถัง (bin) ใดถังหนึ่งได้ ในขณะที่เธรดอื่นทั้งหมดจะต้อง cuckoo ไปยังถังอื่น เมื่อ $b > 1$ เธรดจำนวนมากขึ้นสามารถทำสำเร็จได้ จึงมีสายโซ่คุกคู (cuckoo chain) น้อยลง

ผลของขนาดบักเก็ตต่อรูปแบบการเข้าถึงหน่วยความจำ: การใช้ขนาดบักเก็ตที่ใหญ่ขึ้นไม่เพียงแต่ช่วยปรับปรุง load factor ที่ทำได้เท่านั้น แต่ยังสอดคล้องกับรูปแบบการเข้าถึงหน่วยความจำที่เหมาะสมที่สุดของ GPU ด้วย โดยทั่วไป เราต้องการให้เธรดที่อยู่ติดกันเข้าถึงตำแหน่งหน่วยความจำที่อยู่ติดกัน เพื่อให้เกิดการเข้าถึงหน่วยความจำแบบ*รวมกลุ่ม* (coalesced) เมื่อใช้คีย์ขนาด 4 ไบต์และค่าขนาด 4 ไบต์ และแคชไลน์ของ GPU มีขนาด 128 ไบต์ เราคาดว่าขนาดบักเก็ตที่เหมาะสมที่สุดจากมุมมองของระบบหน่วยความจำคือ 16

เราสรุปว่าการแบ่ง cuckoo hashing เป็นบักเก็ตเป็นแนวคิดที่ดีในแง่ของการลดการเข้าถึงหน่วยความจำที่มีต้นทุนสูงโดยแลกกับการคำนวณที่มีต้นทุนต่ำ การเพิ่มขนาดบักเก็ตช่วยปรับปรุงทั้ง load factor ที่ทำได้และจำนวนการโพรบต่อคีย์ อย่างไรก็ตาม การเพิ่มขนาดบักเก็ตเกินขนาดแคชไลน์จะลดอัตราการประมวลผลของการแทรกและการค้นหาที่ทำได้

B. จำนวนฟังก์ชันแฮช

จำนวนฟังก์ชันแฮช ซึ่งตรงกับจำนวนตำแหน่งที่เป็นไปได้สำหรับรายการใด ๆ มีทั้งผลเชิงบวกและเชิงลบต่อประสิทธิภาพของตารางแฮช

- การเพิ่มจำนวนฟังก์ชันแฮชทำให้บรรลุ load factor สูงได้ ตัวอย่างเช่น ในตารางแฮชคุกคูมาตรฐาน การใช้ฟังก์ชันแฮชสองฟังก์ชันสามารถบรรลุ load factor ได้เพียง 0.5 เท่านั้น การเพิ่มตำแหน่งที่เป็นไปได้อีกเพียงหนึ่งตำแหน่งจะเพิ่ม load factor ที่ทำได้นี้ขึ้นเป็น 0.92 [7] อย่างไรก็ตาม ประโยชน์ของการเพิ่มตำแหน่งมากขึ้นมีเพียงเล็กน้อยเมื่อเกินสามฟังก์ชันแฮช เหตุผลเบื้องหลัง load factor ที่สูงขึ้นนี้คือ การเพิ่มตำแหน่งที่เป็นไปได้มากขึ้นช่วยลดโอกาสที่จะติดอยู่ในวงวน (cycle) ขณะย้ายคีย์ระหว่างบักเก็ตเมื่อเกิดการชนกัน รวมทั้งลดจำนวนการโพรบที่จำเป็นสำหรับการแทรก

- อย่างไรก็ตาม การเพิ่มจำนวนฟังก์ชันแฮชก็เพิ่มงานที่จำเป็นในการค้นหาในตารางแฮชด้วย โดยเฉพาะเมื่อรายการที่ค้นหาไม่มีอยู่ในตารางแฮช ในกรณีนี้ การมีฟังก์ชันแฮช h ฟังก์ชันจำเป็นต้องโพรบตารางแฮช h ครั้ง ก่อนจะสรุปได้ว่ารายการที่ค้นหาไม่มีอยู่

ในตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต เมื่อขนาดบักเก็ตใหญ่ การใช้ฟังก์ชันแฮชสองฟังก์ชันก็เพียงพอที่จะบรรลุ load factor สูง แต่ต้องมีการกระจายภาระระหว่างบักเก็ตที่เป็นไปได้ทั้งสอง แนวทางที่ง่ายกว่าซึ่งแทรกคีย์ลงในบักเก็ตแรกโดยใช้ฟังก์ชันแฮชเพียงสองฟังก์ชันจะลด load factor ที่ทำได้ แต่การเพิ่มฟังก์ชันแฮชอีกหนึ่งฟังก์ชันให้กับแนวทางง่าย ๆ นี้ทำให้บรรลุ load factor ที่สูงขึ้นได้ นอกจากนี้ การใช้แนวทางการแทรกแบบง่ายช่วยปรับปรุงอัตราการประมวลผลของการแทรก (เมื่อเทียบกับเทคนิคที่กระจายภาระ) แต่ลดอัตราการประมวลผลของการค้นหา โดยเฉพาะอย่างยิ่ง เมื่อจำนวนการค้นหาแบบพบต่ำ การใช้ฟังก์ชันแฮชมากขึ้นจะลดประสิทธิภาพของการค้นหา เนื่องจากการค้นหาต้องโหลดบักเก็ตจำนวนเท่ากับจำนวนฟังก์ชันแฮช โดยทั่วไป เราเลือกใช้ฟังก์ชันแฮช 3 ฟังก์ชันและกลยุทธ์การแทรกแบบง่าย อย่างไรก็ตาม หากเป้าหมายหลักคือการปรับปรุงอัตราการค้นหา การใช้กลยุทธ์การแทรกแบบกระจายภาระและฟังก์ชันแฮชเพียง 2 ฟังก์ชันจะเป็นทางเลือกที่ดีกว่า

C. กลยุทธ์การวาง

แม้ว่าการเพิ่มจำนวนฟังก์ชันแฮชจะช่วยปรับปรุง load factor ที่ทำได้ แต่ก็เพิ่มจำนวนทางเลือกเมื่อวางรายการด้วย ทางเลือกที่ง่ายที่สุดคือเลือกบักเก็ตแรกเสมอ และทำ cuckooing เฉพาะเมื่อบักเก็ตเต็มเท่านั้น แนวทางที่ซับซ้อนกว่าคือเลือกบักเก็ตที่มีภาระน้อยที่สุด (กล่าวคือ power of two hashing) ซึ่งเพิ่มต้นทุนของการแทรกเนื่องจากต้องโหลดบักเก็ตที่เป็นไปได้ทั้งหมดก่อนตัดสินใจว่าจะแทรกลงในบักเก็ตใด ภายหลังเราจะแสดงให้เห็นว่ากลยุทธ์การวางแบบง่ายมีประสิทธิภาพมากกว่ากลยุทธ์การวางแบบกระจายภาระ Iceberg hashing เป็นกลยุทธ์การวางแบบที่สาม

V. รายละเอียดการพัฒนาบน GPU

หัวข้อนี้มุ่งเน้นวิธีที่เราพัฒนาตารางแฮชของเราบน GPU ตารางแฮชแบบต่าง ๆ มีรายละเอียดการพัฒนาร่วมกันประการหนึ่งคือการรองรับบักเก็ตขนาดใด ๆ ก็ได้ เราจะอภิปรายรายละเอียดการพัฒนาสำหรับตารางแฮชที่โปรแกรมเมอร์เป็นผู้กำหนดขนาดบักเก็ตก่อน จากนั้นจึงอภิปรายรายละเอียดเฉพาะของการพัฒนาการแทรกและการค้นหาสำหรับแต่ละเทคนิคแยกกัน การพัฒนาของเรามุ่งเป้าที่คีย์และค่าขนาด 32 บิต เราใช้จำนวนที่มากที่สุดที่จำนวนเต็มไม่มีเครื่องหมาย (unsigned integer) เก็บได้เป็นคีย์ (หรือค่า) เฝ้ายาม (sentinel) ตัวอย่างเช่น คีย์ (หรือค่า) ว่างจะเป็น `std::numeric_limits<uint32_t>::max()`

A. การรองรับบักเก็ตขนาดใด ๆ

ในการพัฒนาตารางแฮชแบบแบ่งเป็นบักเก็ตบน GPU เราต้องสร้างสมดุลระหว่างข้อพิจารณาสองประการ: บักเก็ตที่ใหญ่ขึ้นช่วยลดจำนวนการเข้าถึงบักเก็ตที่จำเป็นสำหรับการค้นหาและการแทรก แต่บักเก็ตที่ใหญ่ขึ้นก็ค้นหาและจัดการได้ซับซ้อนกว่า อย่างไรก็ตาม บักเก็ตที่ใหญ่ขึ้นเป็นแนวคิดที่สมเหตุสมผลสำหรับการพัฒนาบน GPU ด้วยเหตุผลสองประการต่อไปนี้

- ต้นทุนของการเข้าถึงเวิร์ด (word) เดียวในแคชไลน์ของ GPU นั้น ในการประมาณอันดับแรก (to first order) เท่ากับการเข้าถึงแคชไลน์ทั้งไลน์ ดังนั้น หากขนาดบักเก็ตไม่ใหญ่กว่าแคชไลน์ เราสามารถเข้าถึงบักเก็ตทั้งบักเก็ตด้วยต้นทุนหน่วยความจำเท่ากับการเข้าถึงรายการเพียงรายการเดียว
- แม้ว่าเราจะต้องทำงานมากขึ้นต่อการค้นหาหรือการแทรกหนึ่งครั้งเมื่อใช้บักเก็ตขนาดใหญ่ขึ้น แต่เรายังคงถูกจำกัดด้วยหน่วยความจำ (memory-bound) ดังนั้นงานที่เพิ่มขึ้นจึงไม่ส่งผลต่อประสิทธิภาพของเรา นอกจากนี้ ด้วยนามธรรม (abstraction) cooperative groups ของ CUDA เราสามารถมอบหมายกลุ่มของเธรดให้กับการค้นหาหรือการแทรกแต่ละครั้ง และใช้กลุ่มนั้นเข้าถึงบักเก็ตทั้งบักเก็ต โดยสื่อสารระหว่างเธรดในกลุ่มได้อย่างมีประสิทธิภาพ

ความท้าทายของบักเก็ตขนาด $b > 1$ คือการพัฒนาการดำเนินการที่เกี่ยวข้องกับบักเก็ตทั้งบักเก็ตให้มีประสิทธิภาพ เช่น การค้นหารายการหนึ่ง ๆ ในบักเก็ต การหาช่องว่างในบักเก็ต และการเลือกรายการที่จะนำออกจากบักเก็ต วิธีหนึ่งในการพัฒนาสิ่งนี้คือการรักษาโครงสร้างข้อมูล (เมทาดาทา (metadata)) สำหรับแต่ละบักเก็ต ซึ่งอาจเก็บ เช่น บิตมาสก์ (bitmask) ของช่องที่เต็ม/ว่างในบักเก็ต หรือดัชนีของช่องว่างถัดไป แต่เราเลือกที่จะคำนวณข้อมูลที่จำเป็นทั้งหมดขณะทำงาน (on the fly) เมื่อต้องการใช้ (โดยไม่ใช้เมทาดาทาใด ๆ) เพราะเหตุใด? ประการแรก โครงสร้างข้อมูลใด ๆ จะใช้พื้นที่ในบักเก็ตและลด load factor ของเรา ประการที่สอง เราไม่เพียงมีขีดความสามารถในการคำนวณเหลือเฟือเท่านั้น แต่การคำนวณของเราในส่วนนี้ยังมีประสิทธิภาพสูงมาก เนื่องจากใช้ฟังก์ชันภายใน (intrinsic) ระดับ warp (ระดับ cooperative group)

รายละเอียดการพัฒนาบน GPU: การพัฒนาของเราใช้ cooperative groups เพื่อให้ควบคุมขนาดบักเก็ตได้อย่างยืดหยุ่น ในขณะที่ใช้ประโยชน์จากฟังก์ชันภายในสำหรับการสื่อสารระดับไทล์ (tile) ระหว่างเธรดในไทล์ API ของ cooperative groups โดยเฉพาะกลุ่มแบบระบุชัดเจน (explicit group) `tiled_partition` มีฟังก์ชันระดับไทล์ เช่น `ballot` ซึ่งประเมินเพรดิเคต (predicate) และคืนค่าจำนวนเต็มที่มีบิตถูกตั้งค่าสำหรับผลการประเมินเพรดิเคตของแต่ละเธรด และ `shfl` ซึ่งกระจาย (broadcast) ตัวแปรไปยังเธรดทั้งหมดในไทล์

เราจะอภิปรายรายละเอียดการพัฒนาสำหรับการแทรกแบบร่วมมือกันในระดับไทล์ (tile-cooperative) ก่อน การแทรกแบบร่วมมือกันทำให้สามารถทำให้การดำเนินการทั้งหมดภายในไทล์เป็นลำดับ แล้วการดำเนินการแบบลำดับแต่ละครั้งสามารถอ่านบักเก็ตแบบรวมกลุ่มได้ (ซึ่งเป็นรูปแบบทั่วไปของกลยุทธ์การแบ่งปันงานแบบร่วมมือกันระดับ warp (warp-cooperative work sharing) ของ Ashkiani [3]) รายการโค้ดที่ 1 แสดงการสร้างไทล์ในฟังก์ชันการแทรกแบบร่วมมือกัน ในบรรทัดที่ 3 เราสร้างบล็อกเธรดที่จะบรรจุไทล์ของเราก่อน จากนั้นเราสามารถใช้บล็อกเธรดสร้างไทล์ที่มีขนาดเท่ากับขนาดบักเก็ต (ซึ่งทราบค่า ณ เวลาคอมไพล์) ในบรรทัดที่ 4 ตอนนี้เราสามารถใช้ไทล์และดำเนินการต่าง ๆ เพื่อสอบถามลำดับ (rank) ของเธรดในไทล์ (บรรทัดที่ 5) หรือประเมินเพรดิเคตเพื่อสร้างคิว (queue) ของเธรด

```
bool cooperative_insert(bool to_insert, pair_type pair,
    pair_type* table){
  //Construct the work tile
  cg::thread_block thb = cg::this_thread_block();
  auto tile = cg::tiled_partition<bucket_size>(thb);
  auto thread_rank = tile.thread_rank();
  bool success = true;
  //Perform the insertions
  while(uint32_t work_queue = tile.ballot(to_insert)){
    auto cur_lane = __ffs(work_queue) - 1;
    auto cur_pair = tile.shfl(pair, cur_lane);
    auto cur_result = insert(tile, cur_pair, table);
    if(tile.thread_rank() == cur_lane){
      to_insert = false;
      success = cur_result;
    }
  }
  return success;
}
```

รายการโค้ดที่ 1: การแทรกแบบร่วมมือกันระดับไทล์ (tile-wide cooperative insertion)

ทั้งหมดที่จะดำเนินการในภายหลัง (บรรทัดที่ 8) บรรทัดที่ 9 แสดงการหารายการถัดไปในคิว และด้วยข้อมูลนี้ คู่ที่สัมพันธ์กับเลน (lane) นั้นจะถูกกระจายไปยังเธรดทั้งหมดในไทล์ในบรรทัดที่ 10 ในบรรทัดที่ 11 คู่นั้นจะถูกแทรกลงในตารางแฮช สุดท้าย ในบรรทัดที่ 12 เฉพาะเลนปัจจุบันเท่านั้นที่นำรายการออกจากคิวโดยตั้งค่า `to_insert` เป็น `false` (บรรทัดที่ 13) และเขียนผลลัพธ์กลับไปยังรีจิสเตอร์ (register) ของตนเองในบรรทัดที่ 14 การค้นหาทำในลักษณะคล้ายกัน โดยมีความแตกต่างเพียงว่าผลลัพธ์จะเป็นค่าของคีย์ (หรือคีย์เฝ้ายามหากไม่พบ) และเรียกการดำเนินการ find แทนการแทรก

ต่อไปเราอภิปรายการดำเนินการร่วมบนบักเก็ตที่แสดงในรายการโค้ดที่ 2 ในบรรทัดที่ 3 เราสร้างโครงสร้างบักเก็ตโดยใช้พอยน์เตอร์ไปยังบักเก็ตและไทล์ของ cooperative group ในการคำนวณภาระของบักเก็ต (บรรทัดที่ 7) เราใช้การดำเนินการ `ballot` เพื่อเติมบิตลงในจำนวนเต็มไม่มีเครื่องหมาย (หนึ่งบิตสำหรับแต่ละเธรดในไทล์) โดยบิตจะถูกตั้งค่าเมื่อคีย์ในบักเก็ตไม่ใช่คีย์เฝ้ายาม เมื่อได้จำนวนเต็มนี้แล้ว เราสามารถนับจำนวนคีย์ที่ใช้ได้ในบักเก็ตโดยใช้ฟังก์ชันภายในนับจำนวนบิต (population count) (`__popc`) นอกจากนี้ เรายังสามารถใช้ภาระนี้เพื่อหาตำแหน่งที่ว่างถัดไปสำหรับการแทรกในบักเก็ตได้ด้วย

ในการค้นหา (บรรทัดที่ 12) เราเปรียบเทียบคีย์ที่ค้นหากับคีย์ที่อยู่ในบักเก็ตก่อน แล้วตามด้วยคำสั่ง `ballot` เพื่อบีบอัดผลการเปรียบเทียบให้เป็นจำนวนเต็มไม่มีเครื่องหมายเพียงตัวเดียว โดยการหาบิตแรกที่ถูกตั้งค่าในผลลัพธ์ (โดยใช้ `__ffs`) เราสามารถระบุตำแหน่งของคีย์ได้ สังเกตว่าดัชนีของบิตที่มีนัยสำคัญน้อยที่สุดคือ 1 ในขณะที่ผลลัพธ์เป็นศูนย์บ่งบอกว่าคีย์ไม่มีอยู่ สุดท้าย เรากระจายผลลัพธ์ไปยังเธรดทั้งหมดในไทล์

วิธีหนึ่งในการแทรกคือการสลับคู่เฝ้ายามที่ว่างกับคู่ใหม่ (บรรทัดที่ 19) โดยใช้คำสั่งเปรียบเทียบและสลับ (compare-and-swap: CAS) มีเพียงเธรดเดียวที่ดำเนินการ CAS และพยายามสลับคู่เฝ้ายามที่ว่างกับคู่ใหม่ จากนั้นเรากระจายคู่ที่ถูกสลับออกไปยังเธรดทั้งหมดในไทล์ เราจะสรุปได้ว่าการแทรกสำเร็จก็ต่อเมื่อคู่ที่ถูกสลับออกเป็นคู่ว่างจริง ๆ เท่านั้น อีกวิธีหนึ่งในการแทรกคือการสลับคู่เก่ากับคู่ใหม่ (บรรทัดที่ 27) ในทำนองเดียวกัน มีเพียงเธรดเดียวที่ดำเนินการสลับ จากนั้นเรากระจายคู่ที่ถูกสลับไปยังเธรดทั้งหมดในไทล์

```
struct bucket{
  // Constructor to load the key-value pair of the bucket
  bucket(pair_type* ptr, const tile_type& tile) : ptr_(
      ptr), tile_(tile){
    lane_pair_ = ptr[tile_.thread_rank()];
  }
  // Compute the load of the bucket
  int compute_load() const {
    auto load_bitmap = tile_.ballot(lane_pair_.key !=
        EMPTY_KEY);
    return __popc(load_bitmap);
  }
  // Find the value associated with a key
  value_type find_key_value(const key_type key) {
    bool key_exist = (key == lane_pair_.key);
    int key_lane = __ffs(tile.ballot(key_exist));
    if(key_lane == 0) return EMPTY_KEY;
    return tile_.shfl(lane_pair_.value, key_lane - 1);
  }
  // Perform an insertion using compare-and-swap
  pair_type cas_at_location(const pair_type& pair, const
      int location) {
    pair_type old_pair;
    if(tile_.thread_rank() == elected_lane_){
      old_pair = atomicCAS(ptr_ + location, EMPTY_PAIR,
          pair);
    }
    return tile_.shfl(old_pair, elected_lane_);
  }
  // Perform an exchange operation
  pair_type exch_at_location(const pair_type& pair, const
      int location) {
    pair_type old_pair;
    if(tile_.thread_rank() == elected_lane_){
      old_pair = atomicExch(ptr_ + location, pair);
    }
    return tile_.shfl(old_pair, elected_lane_);
  }
  private:
  pair_type* ptr_;
  pair_type lane_pair_;
  const tile_type tile_;
  const int elected_lane_ = 0;
}
```

รายการโค้ดที่ 2: การพัฒนาบักเก็ตขนาดเท่าไทล์ (tile-sized bucket)

B. *ตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต*

การแทรก: รายการโค้ดที่ 3 แสดงรายละเอียดการพัฒนาการแทรกในตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต เราเริ่มต้นด้วยการคำนวณบักเก็ตแรกที่เราจะพยายามแทรก (บรรทัดที่ 2) จากนั้นเราคำนวณภาระของบักเก็ต และหากบักเก็ตเต็มและเรายังไม่ถึงจำนวนสายโซ่คุกคูสูงสุด เราจะสลับคู่ใหม่ของเรากับคู่หนึ่งที่สุ่มเลือกจากบักเก็ต (บรรทัดที่ 11) จากนั้นเราเพิ่มค่าตัวนับจำนวนการสลับที่ทำไปแล้ว (บรรทัดที่ 19) ในการสร้างจำนวนสุ่ม เราใช้ Xorshf ของ Marsaglia [16] ซึ่งกำหนดค่าเริ่มต้นครั้งเดียวต่อไทล์ เราได้อภิปรายกรณีที่บักเก็ตแรกเต็มและต้องทำ cuckooing ไปแล้ว ทีนี้หากบักเก็ตมีช่องว่าง เราจะพยายามแทรกคู่ลงในช่องนั้น เราดำเนินการเปรียบเทียบและสลับแบบอะตอมิก (บรรทัดที่ 21) แล้วเปรียบเทียบคู่ที่ถูกสลับออกกับคู่เฝ้ายามที่ว่างและสิ้นสุดหากการแทรกสำเร็จ (บรรทัดที่ 22) หากการแทรกไม่สำเร็จ

เราจะพยายามแทรกอีกครั้งจนกว่าจะสำเร็จหรือถึงจำนวนสายโซ่คุกคูสูงสุด (บรรทัดที่ 10)

```
bool insert(const tile_type tile, const pair_type pair,
     pair_type* table){
 auto bucket_id = hf0(pair.key) % num_buckets;
 uint32_t cuckoo_counter = 0;
 random_number_generator rng(0, bucket_size);
 bool success = false;
 do{
  auto cur_bucket = bucket(hash_table + bucket_id *
       bucket_size, tile);
  auto load = cur_bucket.compute_load();
  if(load == bucket_size){
   if(cuckoo_counter == max_cuckoo_chains) return false;
   pair = cur_bucket.exch_at_location(pair, rng());
   auto prev_bucket = bucket;
   auto bucket0 = hf0(pair.key) % num_buckets;
   auto bucket1 = hf1(pair.key) % num_buckets;
   auto bucket2 = hf2(pair.key) % num_buckets;
   bucket = bucket0;
   bucket = prev_bucket == bucket1 ? bucket2 : bucket;
   bucket = prev_bucket == bucket0 ? bucket1 : bucket;
   cuckoo_counter++;
  }else{
    auto old_pair = cur_bucket.insert_at_location(pair,
         load);
    success = old_pair == EMPTY_PAIR;
   }
 }while(!success);
 return false;
}
```

รายการโค้ดที่ 3: การแทรกในตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต

การค้นหา (find): รายการโค้ดที่ 4 แสดงการพัฒนาการดำเนินการ find ในตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต ในการประเมินการค้นหา เราต้องตรวจสอบ 3 บักเก็ตที่สัมพันธ์กับฟังก์ชันแฮชสามฟังก์ชัน เราตรวจสอบบักเก็ตทั้งสามนี้แบบลำดับภายในไทล์ ในแต่ละครั้ง เราประเมินฟังก์ชันแฮช (บรรทัดที่ 6, 7 หรือ 8) จากนั้นเราโหลดบักเก็ต (บรรทัดที่ 9) เราค้นหาค่าของคีย์ (หากมีอยู่) ในบักเก็ต (บรรทัดที่ 10) และตรวจสอบด้วยว่าบักเก็ตมีคีย์ว่างหรือไม่ (บรรทัดที่ 12) เราสิ้นสุดเมื่อตรวจสอบฟังก์ชันแฮชครบทั้งสามฟังก์ชันแล้ว หรือเมื่อบักเก็ตใดบักเก็ตหนึ่งมีคีย์ว่าง

C. กลยุทธ์การวาง: พลังของสองทางเลือก (หรือมากกว่า)

การแทรก: ต่อไปเราอธิบายรายละเอียดการพัฒนาการแทรกแบบ power-of-two-choices ดังที่แสดงในรายการโค้ดที่ 5 เราประเมินบักเก็ตที่สัมพันธ์กับฟังก์ชันแฮชสองฟังก์ชันก่อน (กล่าวคือ สองทางเลือก) (บรรทัดที่ 2 และ 3) หลังจากคำนวณภาระแล้ว หากบักเก็ตทั้งสองเต็ม การแทรกจะล้มเหลว (บรรทัดที่ 10) มิฉะนั้น เราจะหาบักเก็ตที่มีภาระต่ำกว่า (ตัดสินกรณีเสมอกันโดยเลือกฟังก์ชันแฮชแรก) แล้วแทรกลงในบักเก็ตนั้น (บรรทัดที่ 11, 12 และ 13) สุดท้าย เราพยายามดำเนินการเปรียบเทียบและสลับแบบอะตอมิกเพื่อสลับคู่ว่างตัวสุดท้ายกับคู่ใหม่ และหากการดำเนินการสำเร็จ การแทรกก็สำเร็จ มิฉะนั้น เราจะทำกระบวนการซ้ำจนกว่าจะแทรกคู่ได้ หรือตำแหน่งที่เป็นไปได้ทั้งสองเต็ม

การค้นหา (find): การค้นหาใน BP2HT คล้ายกับใน BCHT (รายการโค้ดที่ 4) โดยมีความแตกต่างเพียงสองประการ: 1) เราต้องตรวจสอบ

```
value_type find(const key_type key, pair_type* table){
 value_type result = EMPTY_VALUE;
 const int num_hash_functions = 3;
 for(int i = 0; i < num_hash_functions && result ==
      EMPTY_VALUE; i++){
    uint32_t bucket_id;
    if(i == 0) bucket_id = hf0(key) % num_buckets;
    else if (i == 1) bucket_id = hf1(key) % num_buckets;
    else bucket_id = hf2(key) % num_buckets;
    auto cur_bucket = bucket(hash_table + bucket_id *
         bucket_size, tile);
    auto result = cur_bucket.find_key_value(key);
    if(result == EMPTY_VALUE && i != num_hash_functions){
        if(cur_bucket.compute_load() != bucket_size)
            return result;
    }
 }
 return result;
}
```

รายการโค้ดที่ 4: การค้นหา (find) ในตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต

```
bool insert(const tile_type tile, const pair_type pair){
  auto bucket0_id = hf0(pair.key) % num_buckets;
  auto bucket1_id = hf1(pair.key) % num_buckets;
  bool keep_trying = true;
  do{
    auto bucket0 = bucket(hash_table + bucket0_id *
         bucket_size, tile);
    auto bucket1 = bucket(hash_table + bucket0_id *
         bucket_size, tile);
    auto load0 = bucket0.compute_load();
    auto load1 = bucket1.compute_load();
    if(load0 == bucket_size && load1 == bucket_size)
         return false;
    auto bucket_choice = load0 <= load1 ? bucket0 :
         bucket1;
    auto load_choice = load0 <= load1 ? load0 : load1;
    keep_trying = bucket_choice.cas_at_location(pair,
         load_choice, 0) != EMPTY_PAIR;
  }while(keep_trying);
  return true;
}
```

รายการโค้ดที่ 5: การแทรกแบบ power of two

เพียงสองบักเก็ต และ 2) เราจะสิ้นสุดได้ก็ต่อเมื่อตรวจสอบทั้งสองบักเก็ตแล้วเท่านั้น (กล่าวคือ ไม่มีการออกก่อนกำหนดดังในบรรทัดที่ 12)

D. กลยุทธ์การวาง: Iceberg hashing

การแทรก: รายการโค้ดที่ 6 แสดงการพัฒนาการแทรกในตารางแฮชไอซ์เบิร์ก เราเริ่มต้นด้วยการคำนวณบักเก็ตหลักและภาระของบักเก็ตนั้น (บรรทัดที่ 2 และ 5) เมื่อทราบภาระแล้ว หากภาระมากกว่าค่าขีดเริ่ม เราจะตรวจสอบฟังก์ชันแฮชรองสองฟังก์ชันและบักเก็ตของฟังก์ชันเหล่านั้น เราคำนวณดัชนีของบักเก็ตและภาระของบักเก็ตเหล่านั้น (บรรทัดที่ 7 ถึง 12) สังเกตว่าหากบักเก็ตรองทั้งสองเต็ม เราจะใช้บักเก็ตหลัก มิฉะนั้น เราจะหาบักเก็ตที่มีภาระน้อยที่สุดและใช้บักเก็ตนั้นสำหรับการแทรก (บรรทัดที่ 14 และ 15) หากหลังจากตรวจสอบบักเก็ตที่เป็นไปได้ทั้งสามแล้วไม่มีที่ว่างสำหรับการแทรก การแทรกจะล้มเหลว (บรรทัดที่ 18) สุดท้าย เราพยายามแทรก (บรรทัดที่ 19) และหากคู่ที่ถูกคืนกลับมาเป็นคู่เป้าหมาย การแทรกก็สำเร็จ มิฉะนั้น เรา

วิธี	Load factor	การโพรบเมื่อแทรก	การโพรบเมื่อค้นหา	ความเสถียร
1CHT	0.88	≈ 2.8	สูงสุด 4	ไม่มี
BCHT	0.98	≈ 1.8	สูงสุด 3	ไม่มี
BP2HT	0.92	2	สูงสุด 2	มี
IHT	0.92	1 หรือ 3	สูงสุด 3	มี

ตารางที่ II: คุณสมบัติของตารางแฮชแบบต่าง ๆ 1CHT ใช้ฟังก์ชันแฮช 4 ฟังก์ชัน ในขณะที่ BCHT ใช้ฟังก์ชันแฮชเพียง 3 ฟังก์ชัน BP2HT ใช้สองทางเลือก (กล่าวคือ ฟังก์ชันแฮชสองฟังก์ชัน) ในขณะที่ IHT ใช้ฟังก์ชันแฮชหลักหนึ่งฟังก์ชันและฟังก์ชันแฮชรองสองฟังก์ชัน

จะทำกระบวนการซ้ำจนกว่าการแทรกจะสำเร็จ หรือตำแหน่งที่เป็นไปได้ทั้งสามเต็ม

```
bool insert(const tile_type tile, const pair_type pair,
    pair_type* table){
  auto primary_bucket_id = hfp(pair.key) % num_buckets;
  while(true){
    auto bucket = bucket(table + primary_bucket_id *
        bucket_size, tile);
    auto load = compute_load();
    if(primary_bucket_load > threshold){
      auto bucket0_id = hf0(key) % num_buckets;
      auto bucket1_id = hf1(key) % num_buckets;
      auto bucket_0 = bucket(table + bucket0_id *
          bucket_size, tile);
      auto bucket_1 = bucket(table + bucket_id *
          bucket_size, tile);
      auto load0 = bucket0.compute_load();
      auto load1 = bucket1.compute_load();
      if(load0 != bucket_size && load1 != bucket_size){
        bucket = load0 <= load1 ? bucket0 : bucket1;
        load = load0 <= load1 ? load0 : load1;
      }
    }
    if(insertion_location == bucket_size) return false;
    if(bucket.cas_at_location(pair, load) == EMPTY_PAIR){
      return true;
    }
  }
  return false;
}
```

รายการโค้ดที่ 6: การแทรกแบบ iceberg

การค้นหา (find): เช่นเดียวกับการดำเนินการ find ของ BP2HT การค้นหาใน IHT ไม่สามารถออกก่อนกำหนดได้ แต่ต้องตรวจสอบบักเก็ตทั้งสาม (บักเก็ตหลักและบักเก็ตรองทั้งสอง) เว้นแต่การค้นหาจะสิ้นสุดเมื่อพบค่าของคีย์ในบักเก็ตแรก ๆ

VI. ข้อเสนอแนะระดับสูงในการเลือกตารางแฮช

ผู้ออกแบบตารางแฮชอาจให้ความสำคัญกับ load factor สูงสุดที่ทำได้ อัตราการแทรก อัตราการค้นหา หรือการผสมผสานของสิ่งเหล่านี้ ตารางที่ II สรุปประสิทธิภาพของแต่ละเทคนิคตามลำดับความสำคัญที่แตกต่างกัน ต่อไปเราจะอภิปรายทางเลือกที่เราเลือกและสัญชาตญาณเบื้องหลังสำหรับลำดับความสำคัญแต่ละแบบ

A. *Load factor สูง*

a) ทางเลือก: BCHT บรรลุ load factor สูงที่สุด ตามด้วย IHT และ BP2HT แล้วจึงเป็น 1CHT

b) เพราะเหตุใด?: BCHT สามารถย้ายคีย์ระหว่างบักเก็ตได้จนกว่าจะสำเร็จหรือถึงความยาวสายโซ่คุกคูสูงสุดที่อนุญาต แต่ IHT มีตำแหน่งที่เป็นไปได้สำหรับการแทรกเพียง 3 ตำแหน่ง ในขณะที่ BP2HT มีตำแหน่งที่เป็นไปได้เพียงสองตำแหน่ง 1CHT มี 4 ตำแหน่ง แต่แต่ละตำแหน่งเก็บได้เพียงหนึ่งรายการเท่านั้น การเพิ่มจำนวนฟังก์ชันแฮชสำหรับเทคนิคใด ๆ เหล่านี้ให้ load factor สูงขึ้น แต่ลดอัตราการประมวลผลของการดำเนินการแทรกและการค้นหา

B. *อัตราการประมวลผลของการแทรก*

a) ทางเลือก: BCHT ให้อัตราการประมวลผลสูงที่สุด ตามด้วย IHT, BP2HT แล้วจึงเป็น 1CHT

b) เพราะเหตุใด?: รูปแบบการเข้าถึงหน่วยความจำของ 1CHT ไม่เหมาะสมที่สุด เนื่องจากเข้าถึงเพียงคีย์เดียว (และค่าของคีย์นั้น) จากแคชไลน์ขนาด 128 ไบต์ จึงใช้ประโยชน์จากแคชไลน์และแบนด์วิดท์เพียง 1/16 นอกจากนี้ ขนาดบักเก็ตที่เล็กยังเพิ่มจำนวนการโพรบเฉลี่ยต่อคีย์ ในทางตรงกันข้าม เทคนิคแบบแบ่งเป็นบักเก็ตใช้ประโยชน์จากแคชไลน์ทั้งไลน์ ในบรรดาเทคนิคแบบแบ่งเป็นบักเก็ต BP2HT ถ่ายโอนหน่วยความจำมากกว่า เนื่องจากต้องโหลดบักเก็ตทั้งสองก่อนตัดสินใจว่าจะเขียนลงบักเก็ตใด ในทำนองเดียวกัน IHT ตรวจสอบบักเก็ตหลัก และหากความจุของบักเก็ตหลักเกินค่าขีดเริ่ม ก็จะตรวจสอบบักเก็ตรองทั้งสอง ที่ load factor ต่ำ บักเก็ตรองจะถูกตรวจสอบน้อยลง และต้องการพฤติกรรมด้านหน่วยความจำใกล้เคียงกับ BCHT แต่ที่ load factor สูง การตรวจสอบบักเก็ตรองมีแนวโน้มมากขึ้น และการถ่ายโอนหน่วยความจำที่จำเป็นจะสูงกว่า BCHT

C. *อัตราการประมวลผลของการค้นหา*

a) ทางเลือก: BCHT ให้อัตราการประมวลผลสูงที่สุด ตามด้วย IHT, BP2HT แล้วจึงเป็น 1CHT

b) เพราะเหตุใด?: เช่นเดียวกับการแทรก การเข้าถึงหน่วยความจำขนาดเท่าแคชไลน์ของ BCHT ทำให้มีประสิทธิภาพเหนือกว่า 1CHT IHT ให้ความสำคัญกับการแทรกในบักเก็ตหลักมากกว่าบักเก็ตรองทั้งสอง ซึ่งทำให้ตรวจสอบบักเก็ตน้อยกว่า BP2HT แต่มากกว่า BCHT อย่างไรก็ตาม หากคีย์ที่ค้นหาไม่มีอยู่ในตารางแฮช IHT จะตามหลัง BP2HT เนื่องจาก IHT ต้องอ่าน 3 บักเก็ตเสมอ เทียบกับ 2 บักเก็ตใน BP2HT 1CHT สามารถมีประสิทธิภาพดีกว่า BP2HT และ IHT ที่ใช้บักเก็ตขนาดใหญ่ (เช่น ขนาดบักเก็ต 32) ต้นทุนของการโหลดแคชไลน์เพิ่มเติมเหล่านี้สูงกว่าต้นทุนที่ 1CHT ต้องใช้ (1 แคชไลน์ต่อฟังก์ชันแฮช กล่าวคือ 4 แคชไลน์)

D. *ความเสถียร*

a) ทางเลือก: IHT และ BP2HT เป็นตารางแฮชเพียงสองแบบที่รองรับความเสถียร

b) เพราะเหตุใด?: ทั้ง IHT และ BP2HT เขียนคีย์ลงในบักเก็ตจำนวนหนึ่ง ขึ้นอยู่กับจำนวนฟังก์ชันแฮชที่ใช้ และเมื่อวางคีย์ลงในบักเก็ตแล้ว คีย์นั้นจะไม่ถูกย้ายอีกเลย ในทางกลับกัน 1CHT และ BCHT มีอิสระในการย้ายคีย์ระหว่างบักเก็ต ซึ่งหมายความว่าทั้งสองไม่มีความเสถียร การแลกเปลี่ยนในที่นี้คือการแลกเปลี่ยนระหว่างความเสถียรกับ load factor สูงสุดที่ทำได้

VII. ผลการทดลอง

ต่อไปนี้เราจะวิเคราะห์และเปรียบเทียบอย่างละเอียดระหว่างการกำหนดค่าตารางแฮชสามแบบที่เราอธิบายในบทความนี้กับ 1CHT ของ CUDPP ตัวชี้วัดคุณภาพ (figure of merit) ของเราคืออัตราการค้นหา อัตราการสร้าง และ load factor เราประเมินการพัฒนาตารางแฮชทุกแบบ[7] บน GPU NVIDIA TITAN V (Volta) ที่มี DRAM 12 GB และ CPU Intel Xeon E5-2637 โค้ดของเราคอมไพล์ด้วย CUDA 11.1 GPU นี้มีแบนด์วิดท์ DRAM ตามทฤษฎีที่ทำได้ 652.8 GiB/s ผลลัพธ์ทั้งหมดเป็นค่าเฉลี่ยจากการทดลองที่สำเร็จ 10 ครั้ง โดยยอมให้ล้มเหลวได้สูงสุด 50 ครั้ง เว้นแต่จะระบุไว้เป็นอย่างอื่น คีย์ (และค่า) ทั้งหมดเป็นคีย์ขนาด 32 บิต ที่ไม่ซ้ำกัน ซึ่งสร้างแบบสุ่มโดยใช้ `std::mt19937` ซึ่งเป็นเอนจินสร้างจำนวนสุ่มที่อิงอัลกอริทึม Mersenne Twister จากนั้นแปลงด้วย `std::uniform_int_distribution` ให้เป็นจำนวนเต็มไม่มีเครื่องหมายที่กระจายแบบเอกรูป (uniformly distributed) ตั้งแต่ 0 จนถึงจำนวนเต็มไม่มีเครื่องหมายค่าสูงสุด เราใช้ฟังก์ชันแฮช $h(k; a, b) = ((ak + b) \mod p) \mod L$ โดยที่ a และ b เป็นจำนวนเต็มที่สร้างแบบสุ่ม และ p เป็นจำนวนเฉพาะสุ่ม (เราใช้ $p = 4294967291$) และ L คือจำนวนบักเก็ตในตารางแฮช

เราใช้ตารางแฮชรูปแบบต่าง ๆ ต่อไปนี้ในการประเมิน:

1CHT ตารางแฮชคุกคูที่มีขนาดบักเก็ตหนึ่ง ซึ่งตรงตามงานของ Alcantara และคณะ [2] และการพัฒนาใน CUDPP ของพวกเขา [10] เราใช้ฟังก์ชันแฮชสี่ฟังก์ชันเช่นเดียวกับการกำหนดค่าเริ่มต้นของ CUDPP

BCHT ตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ต ซึ่งเราเพิ่ม b คือจำนวนรายการต่อบักเก็ต ให้มีค่ามากขึ้น

BP2HT ตารางแฮชคุกคูแบบ power-of-two-choices ที่แบ่งเป็นบักเก็ต โดยมี $b > 1$

IHT ตารางแฮชไอซ์เบิร์ก ซึ่งกำหนดพารามิเตอร์ด้วยจำนวนรายการต่อบักเก็ต b และค่าขีดเริ่ม t

A. การกำหนดค่าพารามิเตอร์ของตารางแบบแบ่งเป็นบักเก็ตและแบบไอซ์เบิร์ก

เราเริ่มการวิเคราะห์ประสิทธิภาพด้วยการกำหนดค่าพารามิเตอร์ที่เหมาะสมที่สุดสำหรับตารางแฮชแต่ละแบบ สำหรับแต่ละเทคนิคเราประเมินอัตราการประมวลผลของการค้นหาและการแทรกในสองสถานการณ์ ได้แก่ เมื่อจำนวนคีย์คงที่และ load factor เปลี่ยนแปลง และเมื่อ load factor คงที่และจำนวนคีย์เปลี่ยนแปลง เรายังประเมินประสิทธิภาพการค้นหาสำหรับสัดส่วนการค้นหาแบบพบ (สัดส่วนของคีย์ที่ค้นหาซึ่งมีอยู่ในตารางแฮช) สามค่า ได้แก่ 100%, 50% และ 0% เมื่อจำนวนคีย์คงที่ เราใช้ 50 ล้านคีย์เป็นขนาดข้อมูลนำเข้า และเมื่อเราเปลี่ยน load factor เราเลือก load factor สองค่าคือ 0.8 และ 0.9 เนื่องจากการพัฒนาของเราใช้ cooperative groups เป็นองค์ประกอบหลัก ขนาดบักเก็ตจึงเป็นเลขยกกำลังของสองเสมอ โดยมีค่าได้สูงสุดเท่ากับขนาด warp (กล่าวคือ 32) เราหลีกเลี่ยงการใช้ขนาดบักเก็ตที่ใหญ่กว่านี้ (เราคาดว่าจะไม่สามารถแข่งขันด้านประสิทธิภาพได้) เนื่องจากฟังก์ชันการทำงานของ cooperative groups มีข้อจำกัดมากกว่า และมีต้นทุนเพิ่มเติมจากการใช้หน่วยความจำร่วม (shared memory) ในการสื่อสารระหว่างเธรดในไทล์เมื่อดำเนินการระดับไทล์

[7]การพัฒนาของเราเผยแพร่ที่ https://github.com/owensgroup/BGHT

พารามิเตอร์ที่คงที่	b	การแทรก	% ของคีย์ที่ค้นหาซึ่งมีอยู่ในตารางแฮช		
			100%	50%	0%
50M คีย์	1	735.84	2235	1884.83	1496.21
	8	1150.60	3396.57	2645.33	2127.42
	16	**1273.86**	**3642.61**	**2840.57**	**2313.17**
	32	1140.14	2166.94	1739.29	1450.70
load factor 0.8	1	772.08	2313.14	1945.56	1530.62
	8	1309.63	3858.59	3206.12	2695.23
	16	**1425.00**	**4033.52**	**3546.12**	**3155.51**
	32	1243.22	2334.66	2181.45	2050.83
load factor 0.9	1	580.73	1987.13	1646.54	1323.49
	8	1193.83	3502.35	2643.75	2081.43
	16	**1344.38**	**3799.58**	**2921.74**	**2359.95**
	32	1206.40	2260.30	1839.16	1550.71

ตารางที่ III: อัตราการประมวลผลเฉลี่ยของการแทรกและการค้นหา (ที่เปอร์เซ็นต์การค้นหาแบบพบต่าง ๆ) สำหรับ BCHT เมื่อ: 1) จำนวนคีย์คงที่ที่ 50M คีย์ และ 2) load factor คงที่ที่ 0.8 และ 0.9

1) BCHT: รูปที่ 3 แสดงอัตราการประมวลผลสำหรับสถานการณ์ที่จำนวนคีย์คงที่และ load factor เปลี่ยนแปลง ซึ่งแสดงให้เห็นว่าในทุกสถานการณ์ BCHT ที่มีขนาดบักเก็ต 16 มีประสิทธิภาพเหนือกว่าขนาดบักเก็ตทางเลือกอื่นทั้งหมด ผลลัพธ์เหล่านี้ตรงกับที่เราคาดไว้เนื่องจากจากมุมมองของฮาร์ดแวร์ เราต้องการอย่างยิ่งให้ขนาดบักเก็ตตรงกับขนาดแคชไลน์ของ GPU คือ 128 ไบต์ หรือคู่คีย์–ค่าขนาด 4 ไบต์จำนวน 16 คู่ สำหรับทั้งการค้นหาและการแทรก ปัจจัยหลักที่ส่งผลต่อประสิทธิภาพคือ load factor load factor ที่สูงขึ้นทำให้อัตราการประมวลผลของ BCHT ต่ำลง ซึ่งเป็นผลจากจำนวนสายโซ่คุกคู (หรือการขับออก (eviction)) ที่เราต้องทำต่อการแทรกคีย์หนึ่งครั้งเพิ่มขึ้น ปัจจัยเพิ่มเติมที่ลดประสิทธิภาพของอัตราการประมวลผลของการค้นหาคือสัดส่วนของคีย์ที่ค้นหาซึ่งมีอยู่ในตารางแฮช การค้นหาแบบไม่พบต้องตรวจสอบมากกว่าหนึ่งบักเก็ต โดยเฉพาะที่ load factor สูง (กล่าวคือ เมื่อบักเก็ตส่วนใหญ่เต็ม) ซึ่งทำให้ประสิทธิภาพลดลงอย่างรวดเร็วกว่าในสถานการณ์ที่การค้นหาจำนวนมากขึ้นไม่พบคีย์

รูปที่ 4 แสดงการเปรียบเทียบระหว่างขนาดบักเก็ตต่าง ๆ ของ BCHT สำหรับ load factor สองค่า (0.8 และ 0.9) ในรูปนี้ เราเห็นว่าเมื่อตารางแฮชมีขนาดใหญ่พอที่จะเกินขนาดแคช L2 (ในที่นี้ แคช L2 ของเราเก็บได้สูงสุด 589k คู่) อัตราการประมวลผลเกือบคงที่สำหรับทุกขนาดบักเก็ต ตารางที่ III ให้สรุปค่าเฉลี่ยของผลลัพธ์สำหรับขนาดบักเก็ตต่าง ๆ

รูปที่ 5 แสดงจำนวนการโพรบเฉลี่ยสำหรับการดำเนินการต่าง ๆ ของตารางแฮช สำหรับทุกการดำเนินการ เราเห็นข้อได้เปรียบที่ชัดเจนเมื่อใช้ตารางแฮชแบบแบ่งเป็นบักเก็ตเทียบกับ 1CHT ตัวอย่างเช่น จำนวนการโพรบเฉลี่ยสำหรับการแทรกลดลงจาก 2.75 เป็น 1.23 เมื่อขนาดบักเก็ตเพิ่มจากหนึ่งเป็นแปด (ที่ load factor 0.9) การเพิ่มขนาดบักเก็ตเกินแปดให้การปรับปรุงเพียงเล็กน้อย โดยได้จำนวนการโพรบเฉลี่ย {1.11, 1.05} สำหรับขนาดบักเก็ต {16, 32} อย่างไรก็ตาม

พารามิเตอร์ที่คงที่	b	การแทรก	% ของคีย์ที่ค้นหาซึ่งมีอยู่ในตารางแฮช		
			100%	50%	0%
50M คีย์	16	**1058.26**	**3091.04**	**2507.89**	**2123.46**
	32	849.54	1738.48	1410.54	1190.29
load factor 0.8	16	**1100.27**	**3169.81**	**2583.22**	**2194.49**
	32	878.6	1778.99	1446.50	1222.44
load factor 0.9	32	**886.15**	**1785.97**	**1452.68**	**1227**

ตารางที่ IV: อัตราการประมวลผลเฉลี่ยของการแทรกและการค้นหา (ที่เปอร์เซ็นต์การค้นหาแบบพบต่าง ๆ) สำหรับ BP2HT เมื่อ: 1) จำนวนคีย์คงที่ที่ 50M คีย์ และ 2) load factor คงที่ที่ 0.8 และ 0.9

พารามิเตอร์ที่คงที่	b	การแทรก	% ของคีย์ที่ค้นหาซึ่งมีอยู่ในตารางแฮช		
			100%	50%	0%
50M คีย์	16	**1297.18**	**3392.59**	**1922.64**	**1436.26**
	32	1087.01	1946.14	1088.09	803.85
load factor 0.8	16	**1319.87**	**3350.79**	**1941.20**	**1485.75**
	32	1145.62	2042.45	1131.60	825.79
load factor 0.9	32	**1073.92**	**1860.21**	**1075.96**	**829.85**

ตารางที่ V: อัตราการประมวลผลเฉลี่ยของการแทรกและการค้นหา (ที่เปอร์เซ็นต์การค้นหาแบบพบต่าง ๆ) สำหรับ IHT เมื่อ: 1) จำนวนคีย์คงที่ที่ 50M คีย์ และ 2) load factor คงที่ที่ 0.8 และ 0.9

ดังที่เราได้อภิปรายไว้ก่อนหน้านี้ ขนาดบักเก็ต 16 ตรงกับขนาดแคชไลน์ของ GPU จึงให้อัตราการประมวลผลที่ดีกว่า จำนวนการโพรบต่อการค้นหาถูกจำกัดด้วยจำนวนฟังก์ชันแฮชสำหรับทุกขนาดบักเก็ตในสถานการณ์ที่ดีที่สุด (กล่าวคือ เมื่อการค้นหาพบคีย์ในบักเก็ตแรก) จำนวนการโพรบจะใกล้เคียงหนึ่ง เมื่อการค้นหาทั้งหมดเป็นแบบพบ เราเห็นว่าเทคนิคแบบแบ่งเป็นบักเก็ตต้องใช้การโพรบสูงสุด 1.5 ครั้งต่อคีย์ที่ load factor สูงถึง 0.98 อย่างไรก็ตาม ในสถานการณ์ที่เลวร้ายที่สุด (กล่าวคือ load factor สูงและการค้นหาแบบไม่พบ) การค้นหาจะโหลดบักเก็ตที่เป็นไปได้ทั้งหมด ตัวอย่างเช่น เมื่อขนาดบักเก็ตเป็น 16 และที่ load factor 0.99 จำนวนการโพรบเฉลี่ยต่อการค้นหาคือ 2.8

2) BP2HT: เราทดลองกับขนาดบักเก็ตต่าง ๆ สำหรับ BP2HT load factor ที่ทำได้สำหรับขนาดบักเก็ต {8, 16, 32} คือ {0.65, 0.84, 0.92} เราเลือกขนาดบักเก็ตสองขนาดที่บรรลุ load factor สูงและใช้ตลอดการทดลองต่อจากนี้ รูปที่ 6 แสดงประสิทธิภาพของ BP2HT เมื่อจำนวนคีย์คงที่ จะเห็นได้ชัดว่า $b = 16$ มีประสิทธิภาพดีกว่า $b = 32$ ในทุกการทดสอบ แต่ไม่สามารถบรรลุ load factor สูงสุดได้เท่ากับ $b = 32$ เนื่องจากการแทรกใน BP2HT ต้องโหลดเพียงสองบักเก็ต จากนั้นดำเนินการเปรียบเทียบและสลับแบบอะตอมิกในบักเก็ตที่มีภาระน้อยที่สุด ประสิทธิภาพจึงคงที่สำหรับทุก load factor ในทำนองเดียวกัน เมื่อ load factor คงที่ อัตราการประมวลผลก็คงที่สำหรับขนาดข้อมูลนำเข้าขนาดใหญ่ทั้งหมด รูปที่ 7 แสดงผลลัพธ์สำหรับ load factor สองค่า (0.8 และ 0.9) อีกครั้งหนึ่งที่ $b = 16$ มีประสิทธิภาพดีกว่า $b = 32$ แต่ไม่สามารถบรรลุ load factor 0.9 ได้ เราสรุปผลลัพธ์ของ BP2HT ไว้ในตารางที่ IV

การแทรกใน BP2HT ต้องโหลดสองบักเก็ต ดังนั้นจำนวนการโพรบเฉลี่ยจึงเป็นสองโดยไม่ขึ้นกับขนาดบักเก็ตหรือ load factor ในทางกลับกัน การค้นหาต้องใช้การโพรบเพียง 1.33 ครั้งต่อคีย์เมื่อการค้นหาทั้งหมดเป็นแบบพบ กล่าวอีกนัยหนึ่งคือ มีเพียง 33% ของเวลาเท่านั้นที่การค้นหาอ่านบักเก็ตที่สอง เมื่อการค้นหา 50% เป็นแบบพบ จำนวนการโพรบต่อคีย์จะเพิ่มขึ้นเป็น 1.67 ในสถานการณ์ที่เลวร้ายที่สุด (กล่าวคือ การค้นหาทั้งหมดเป็นแบบไม่พบ) BP2HT จะโหลดทั้งสองบักเก็ตก่อนประเมินการค้นหา รูปที่ 8 แสดงจำนวนการโพรบเฉลี่ยสำหรับการดำเนินการแทรกและการค้นหา

3) IHT: เราทดลองกับปริภูมิพารามิเตอร์ของ IHT โดยเปลี่ยนทั้งค่าขีดเริ่ม t และขนาดบล็อก b เรากวาดค่าพารามิเตอร์ (parameter sweep) สำหรับการทดลองแรกที่จำนวนคีย์คงที่ในรูปที่ 9 เราพบว่า IHT ต้องการขนาดบักเก็ตที่ใหญ่ขึ้นเพื่อบรรลุ load factor สูง กล่าวคือ ขนาดบักเก็ต {32, 16, 8} บรรลุ load factor สูงสุด {0.93, 0.86, 0.7} ตามลำดับ โดยทั่วไป เมื่อ load factor เพิ่มขึ้น การแทรก (หรือการค้นหา) จำนวนมากขึ้นต้องเข้าถึงบักเก็ตรองทั้งสอง ซึ่งนำไปสู่ประสิทธิภาพที่ลดลง ระหว่างการค้นหา IHT ต้องตรวจสอบบักเก็ตที่เป็นไปได้ทั้งสาม เว้นแต่การค้นหาจะพบคีย์ในบักเก็ตแรกหรือบักเก็ตที่สอง (สังเกตว่าเมื่อการค้นหาทั้งหมดเป็นแบบไม่พบ การค้นหาต้องตรวจสอบทั้งสามบักเก็ต)

เราใช้ขนาดบักเก็ต 16 และ 32 ตลอดการทดลองของเรา เนื่องจากบรรลุ load factor ที่เทียบเคียงได้กับตารางแฮชทางเลือกอื่น สำหรับทั้งสองขนาดบักเก็ต เราพบว่าเมื่อค่าขีดเริ่มเพิ่มขึ้น อัตราการประมวลผลที่ได้ ไม่ว่าจะเป็นการแทรกหรือการค้นหา ก็เพิ่มขึ้นด้วย เมื่อค่าขีดเริ่มเพิ่มขึ้น การแทรกจำนวนมากขึ้นจะเกิดขึ้นในบักเก็ตหลัก จึงหลีกเลี่ยงการโหลดบักเก็ตรองและประหยัดแบนด์วิดท์ ค่าขีดเริ่มที่ให้ประสิทธิภาพดีที่สุดสำหรับทั้งสองขนาดบักเก็ตอยู่ที่ประมาณ 80% ของขนาดบักเก็ต ตารางที่ V ให้สรุปผลลัพธ์ของเราสำหรับ IHT

รูปที่ 11 แสดงจำนวนการโพรบเฉลี่ยสำหรับขนาดบักเก็ตและค่าขีดเริ่มต่าง ๆ ของ IHT เราพบว่าเมื่อค่าขีดเริ่มเพิ่มขึ้น จำนวนการโพรบลดลงอย่างมีนัยสำคัญ ตัวอย่างเช่น เมื่อแทรกลงใน IHT ที่ load factor 0.86 และขนาดบักเก็ต 16 จำนวนการโพรบเฉลี่ยคือ {2.68, 2.29, 1.89, 1.49} ที่ค่าขีดเริ่ม {20%, 40%, 60%, 80%} การเพิ่มค่าขีดเริ่มหมายถึงการแทรกในบักเก็ตหลักมากขึ้นโดยไม่ต้องอ่านบักเก็ตรองทั้งสอง ซึ่งอธิบายการลดลงอย่างมีนัยสำคัญของจำนวนการโพรบ เช่นเดียวกับข้อสังเกตก่อนหน้านี้เกี่ยวกับจำนวนการโพรบเฉลี่ยของ BCHT การเพิ่มขนาดบักเก็ตจาก 16 เป็น 32 ให้การปรับปรุงจำนวนการโพรบเพียงเล็กน้อย เมื่อการค้นหาทั้งหมดเป็นแบบพบ และที่ load factor 0.86 IHT ที่มีขนาดบักเก็ต 16 และค่าขีดเริ่ม {20%, 40%, 60%, 80%} ทำการโพรบ {2.11, 1.86, 1.6, 1.33} ครั้งต่อคีย์ เมื่อเปอร์เซ็นต์ของการค้นหาแบบพบลดลง การค้นหาจะตรวจสอบบักเก็ตรองมากขึ้น จนตรวจสอบบักเก็ตที่เป็นไปได้ทั้งสามสำหรับ load factor และขนาดบักเก็ตต่าง ๆ

B. การเปรียบเทียบระหว่างการพัฒนาแบบต่าง ๆ

จากผลลัพธ์ก่อนหน้านี้ เราเลือกการกำหนดค่าตารางแฮชสี่แบบต่อไปนี้สำหรับการเปรียบเทียบ: 1) 1CHT ซึ่งเป็นเกณฑ์พื้นฐาน (baseline) 2) BCHT ที่มีขนาดบักเก็ต 16 3) BP2HT ที่มีขนาดบักเก็ต 32 4) IHT ที่มีขนาดบักเก็ต 32 และค่าขีดเริ่ม 80%

1) การเปลี่ยน load factor เมื่อจำนวนคีย์คงที่: รูปที่ 12 แสดงการเปรียบเทียบระหว่างการกำหนดค่าที่แนะนำ ในรูปนี้ เราเห็นว่า BCHT ให้ประสิทธิภาพดีที่สุดทั้งสำหรับการแทรกและการค้นหา BCHT มีข้อได้เปรียบตรงที่สามารถใช้ขนาดบักเก็ต 16 ในขณะที่ยังบรรลุ load factor สูงได้ (กล่าวคือ แต่ละบักเก็ตคือหนึ่งแคชไลน์) ในขณะที่ IHT และ BP2HT ต้องใช้ขนาดบักเก็ต 32 เพื่อให้บรรลุ load factor ที่เทียบเคียงได้ (กล่าวคือ แต่ละบักเก็ตคือสองแคชไลน์) ในทางกลับกัน 1CHT ใช้ข้อมูลเพียง 8 ไบต์จากแคชไลน์ขนาด 128B

ในส่วนของประสิทธิภาพการแทรก ปริมาณการรับส่งข้อมูลหน่วยความจำ (memory traffic) ที่จำเป็นสำหรับ IHT (2 หรือ 6 แคชไลน์ในสถานการณ์ที่ดีที่สุดและเลวร้ายที่สุดตามลำดับ) และ BP2HT (4 แคชไลน์) ทำให้ประสิทธิภาพของทั้งสองต่ำกว่า BCHT (1 ถึง 3 แคชไลน์) ในทางกลับกัน 1CHT มีประสิทธิภาพแย่กว่าทั้ง BCHT และ IHT แต่ดีกว่า BP2HT เฉพาะเมื่อ load factor ถึง 0.72 เท่านั้น ที่ประสิทธิภาพของ BP2HT จะสูงกว่า 1CHT โดยเฉลี่ย อัตราการประมวลผลของการแทรกของ {BCHT, IHT, BP2HT, 1CHT} คือ {1273.86, 1087.01, 849.54, 735.84} Mkey/s

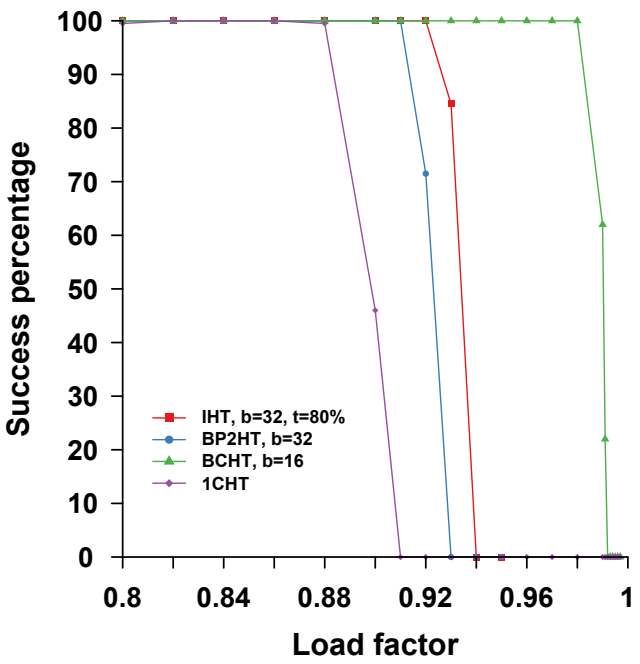
สำหรับการค้นหาที่เป็นแบบพบทั้งหมด (และในสถานการณ์ที่ดีที่สุด) BCHT และ 1CHT ต้องโหลดเพียงหนึ่งแคชไลน์ ในขณะที่ IHT และ BP2HT โหลดสองแคชไลน์ ส่งผลให้อัตราการประมวลผลของ 1CHT สูงกว่าของ IHT และ BP2HT ส่วนอัตราการประมวลผลของ BCHT ยังคงสูงกว่าทางเลือกอื่นทั้งหมด

เมื่ออัตราส่วนของการค้นหาแบบพบลดลง แต่ละเทคนิคต้องตรวจสอบบักเก็ตเพิ่มเติม ในสถานการณ์ที่เลวร้ายที่สุด BCHT โหลด 3 แคชไลน์ 1CHT โหลด 4 แคชไลน์ BP2HT โหลด 4 แคชไลน์ และ IHT โหลด 6 แคชไลน์ ส่งผลให้ลำดับประสิทธิภาพเป็น BCHT (สูงที่สุด) 1CHT, BP2HT และ IHT (ต่ำที่สุด) โดยสรุป อัตราการประมวลผลของการค้นหาเฉลี่ยของ {BCHT, IHT, BP2HT, 1CHT} คือ {3642.61, 1946.14, 1738.48, 2235}, {2840.57, 1088.09, 1410.54, 1884.83} และ {2313.17, 803.85, 1190.29, 1496.21} Mkey/s สำหรับการค้นหาแบบพบทั้งหมด แบบพบ 50% และแบบไม่พบทั้งหมด ตามลำดับ

สังเกตว่าที่ load factor ต่ำ ประสิทธิภาพของ IHT ใกล้เคียงกับ BCHT เมื่อขนาดบักเก็ตเท่ากัน อย่างไรก็ตาม ที่ load factor สูง (ภาระของบักเก็ตเกินค่าขีดเริ่ม) IHT ใช้กลยุทธ์ power of two ซึ่งต้องโหลดบักเก็ตเพิ่มอีกสองบักเก็ต ในทางตรงกันข้าม BCHT โหลดบักเก็ตเพียงเท่าที่จำเป็นในการทำ cuckooing

2) การเปลี่ยนขนาดข้อมูลนำเข้า: ต่อไปเราพิจารณาสถานการณ์ที่เราเปลี่ยนจำนวนคีย์นำเข้าในขณะที่ load factor คงที่ ทุกวิธีให้อัตราการประมวลผลของการแทรกและการค้นหาที่คงที่เมื่อปริมาณคีย์เกินขนาดแคช รูปที่ 13 แสดงประสิทธิภาพของตารางแฮชทั้ง 4 แบบสำหรับ load factor 0.8 และ 0.9 เช่นเดียวกับหัวข้อก่อนหน้า BCHT ให้ประสิทธิภาพดีที่สุด ตามด้วย IHT, BP2HT แล้วจึงเป็น 1CHT โดยสรุป ที่ load factor 0.8 อัตราการประมวลผลของการแทรกของ {BCHT, IHT, BP2HT, 1CHT} คือ {1425, 1145.62, 878.60, 772.08} Mkey/s

สำหรับการค้นหา อัตราการประมวลผลของ {BCHT, IHT, BP2HT, 1CHT} คือ {4033.52, 2042.45, 1778.99, 2313.14}, {3546.12, 1131.6, 1446.50, 1945.56} และ {3155.51, 825.79, 1222.44, 1530.62} สำหรับการค้นหาแบบพบทั้งหมด แบบพบ 50% และแบบไม่พบทั้งหมด ตามลำดับ



รูปที่ 2: การเปรียบเทียบอัตราความสำเร็จระหว่างตารางแฮชรูปแบบที่แนะนำ ข้อมูลนำเข้าคือ 50M คีย์ ทดลองทั้งหมด 200 ครั้ง

C. อัตราความสำเร็จ

ตารางแฮชทั้งหมดที่เราอภิปรายอาจล้มเหลวได้ระหว่างการสร้าง เพื่อประเมินอัตราความสำเร็จของแต่ละเทคนิค เราสร้างตารางแฮชแต่ละแบบโดยใช้ชุดคีย์นำเข้าชุดเดียวกันใน 200 การรันที่แตกต่างกัน สำหรับแต่ละการรัน เราใช้ฟังก์ชันแฮชที่แตกต่างกัน (ด้วยค่าคงที่แบบสุ่ม) และบันทึกความสำเร็จหรือความล้มเหลวของแต่ละการรัน รูปที่ 2 แสดง load factor ที่ทำได้เมื่อเปอร์เซ็นต์ความสำเร็จอยู่ที่ประมาณ 99% BCHT บรรลุ load factor สูงที่สุดที่ 0.98 ตามด้วย IHT และ BP2HT ที่ load factor 0.91 แล้วจึงเป็น 1CHT ที่ load factor 0.88

D. การวิเคราะห์จำนวนการโพรบเฉลี่ย

ปัจจัยหลักที่มีอิทธิพลต่อประสิทธิภาพของการแทรก (หรือการค้นหา) ในตารางแฮชคือจำนวนการโพรบที่แต่ละการดำเนินการต้องทำเพื่อให้การแทรก (หรือการค้นหา) เสร็จสมบูรณ์ เพื่อทำความเข้าใจประสิทธิภาพของแต่ละการดำเนินการ เราติดตั้งเครื่องมือวัด (instrument) ในการพัฒนาของเราเพื่อนับจำนวนการโพรบเฉลี่ยต่อคีย์ที่แต่ละการดำเนินการต้องทำ สังเกตว่าบักเก็ตมีขนาดแตกต่างกันในแต่ละการพัฒนา แต่ในที่นี้การโพรบหนึ่งครั้งเทียบเท่ากับการอ่านบักเก็ตหนึ่งบักเก็ตโดยไม่คำนึงถึงขนาด (การอ่านสองบักเก็ตแทนที่จะเป็นหนึ่งบักเก็ตไม่จำเป็นต้องทำให้ประสิทธิภาพช้าลง 2×) นอกจากนี้ การแทรกใน 1CHT ต้องใช้การสลับแบบอะตอมิกเพียงครั้งเดียว ในขณะที่ในเทคนิคแบบแบ่งเป็นบักเก็ตต้องใช้สองการดำเนินการ: 1) อ่านบักเก็ต 2) ดำเนินการเปรียบเทียบและสลับแบบอะตอมิก (และในบางกรณีใช้การสลับแบบอะตอมิกแทน) เราจะตรวจสอบผลกระทบ

เหล่านี้ในหัวข้อถัดไป แต่ในหัวข้อนี้เราจะพิจารณาเฉพาะจำนวนการโพรบเท่านั้น การพิจารณาเฉพาะจำนวนการโพรบเฉลี่ยทำให้เราเข้าใจประสิทธิภาพของตารางแฮชแต่ละแบบได้โดยไม่ขึ้นกับฮาร์ดแวร์

รูปที่ 14 แสดงจำนวนการโพรบเฉลี่ยสำหรับการแทรกและการค้นหาโดยใช้อัตราส่วนการค้นหาแบบพบต่าง ๆ ในตารางขนาด 50 ล้านคีย์ เราสร้างแต่ละรูปแบบสำหรับ load factor ต่าง ๆ ตั้งแต่ 0.6 จนถึง load factor สูงสุดที่เราทำได้

การแทรก เราพบว่า BCHT มีจำนวนการโพรบเฉลี่ยต่อคีย์ต่ำมาก (สูงสุด 1.43 ที่ load factor 0.99) ซึ่งยากที่วิธีที่แข่งขันใด ๆ จะเอาชนะได้ IHT ก็มีจำนวนการโพรบต่ำใกล้เคียงกัน (สูงสุด 1.46 ที่ load factor 0.92) สาเหตุหลักคือ IHT ต้องอ่านเพียงบักเก็ตหลัก เว้นแต่บักเก็ตหลักจะเกินค่าขีดเริ่ม ซึ่งเมื่อนั้นจะอ่านบักเก็ตเพิ่มอีก 2 บักเก็ต ในสถานการณ์ที่ดีที่สุดสำหรับ IHT คีย์จำนวน threshold% ของคีย์ทั้งหมด (threshold คือค่าขีดเริ่ม) จะต้องอ่านเพียงบักเก็ตหลัก ในขณะที่อีก $(100 - \text{threshold})\%$ จะอ่านบักเก็ตหลักและบักเก็ตรองทั้งสอง BP2HT ต้องโหลดสองบักเก็ตโดยไม่ขึ้นกับ load factor 1CHT ที่มีฟังก์ชันแฮช 4 ฟังก์ชันอ่านสูงสุด 2.75 บักเก็ตต่อการแทรกที่ load factor 0.9

การค้นหา โดยคีย์ทั้งหมดอยู่ในตาราง เมื่อคีย์ที่ค้นหาทั้งหมดมีอยู่ในตารางแฮช จำนวนการโพรบลดลงสำหรับทุกเทคนิค ต่างจากการแทรก การค้นหาไม่มีการดำเนินการแบบอะตอมิกที่อาจล้มเหลว และสำหรับตารางแฮชทุกแบบ การค้นหาไม่จำเป็นต้องเข้าถึงบักเก็ตที่เป็นไปได้ทั้งหมด ตัวอย่างเช่น จำนวนการโพรบเฉลี่ยของ BP2HT ลดลงจาก 2 ในการแทรกเป็น 1.34 สำหรับการค้นหาที่ load factor สูงที่สุด (0.92) ในทำนองเดียวกัน จำนวนการโพรบเฉลี่ยของ BCHT, IHT และ 1CHT ลดลงเป็น 1.38, 1.32 และ 2.26 ที่ load factor สูงสุดที่ทำได้ตามลำดับ

การค้นหา โดยคีย์ไม่ได้อยู่ในตารางทั้งหมด อย่างไรก็ตาม เมื่ออัตราส่วนของการค้นหาแบบพบลดลง (กล่าวคือ มีคีย์ที่ไม่อยู่ในตารางมากขึ้น) จำนวนการโพรบเฉลี่ยจะเพิ่มขึ้น และอาจสูงถึงจำนวนฟังก์ชันแฮชใน 1CHT และ BCHT หรือ 2 และ 3 ใน BP2HT และ IHT ตามลำดับ สังเกตว่าต่างจาก 1CHT และ BCHT ตรงที่ IHT และ BP2HT ไม่สามารถออกก่อนกำหนดได้หากบักเก็ตที่เป็นไปได้บักเก็ตใดบักเก็ตหนึ่งมีช่องว่าง ที่อัตราส่วนการค้นหาแบบพบ 0% นั้น {BCHT, IHT, BP2HT, 1CHT} มีจำนวนการโพรบสูงสุด {2.80, 3, 2, 3.44} ครั้งต่อการค้นหา

ผลลัพธ์ที่เราแสดงในหัวข้อก่อนหน้านี้ยืนยันและเสริมข้อเสนอแนะระดับสูงในการเลือกตารางแฮชของเราในหัวข้อที่ VI

E. การวิเคราะห์อัตราการประมวลผลที่ทำได้และการถ่ายโอนหน่วยความจำ

เพื่อประเมินการพัฒนาของเราและเปรียบเทียบกับความเร็วสูงสุดทางทฤษฎี (speed-of-light) ในอุดมคติของ GPU เราวัดตัวชี้วัดสองแบบ: 1) อัตราการประมวลผลของ DRAM ที่ทำได้ และ 2) จำนวนเซกเตอร์ DRAM (นิยามไว้ด้านล่าง) เฉลี่ยต่อคีย์ (การแทรกหรือการค้นหา) รูปที่ 15 แสดงผลจากเครื่องมือวิเคราะห์ประสิทธิภาพ (profiler) สำหรับการวิเคราะห์เหล่านี้ สำหรับการทดลองนี้ เราใช้จำนวนคีย์ขนาดใหญ่ (200M คีย์) เพื่อหลีกเลี่ยงผลของแคช และเพื่อความเป็นธรรม เราใช้ขนาดบักเก็ตเดียวกันสำหรับตารางแฮชทุกแบบ

อัตราการประมวลผลที่ทำได้: สำหรับตารางแฮชแบบแบ่งเป็นบักเก็ตทั้งหมด การค้นหาทำได้มากกว่า 500 GiB/s (มากกว่า 75.5% ของแบนด์วิดท์ที่ทำได้) 1CHT ทำได้เพียงประมาณ 267 GiB/s ในทางกลับกัน การแทรกทำอัตราการประมวลผลได้ต่ำกว่า (ระหว่าง 220 ถึง 300 GiB/s) ในตารางแฮชแบบแบ่งเป็นบักเก็ต และ 141 GiB/s ใน 1CHT

การถ่ายโอนหน่วยความจำ: เพื่อทำความเข้าใจว่าเหตุใดการแทรกจึงช้ากว่าการค้นหา เราวัดจำนวนเซกเตอร์ DRAM เฉลี่ยต่อคีย์ (หน่วยการวัดของ profiler สำหรับการดำเนินการโหลดและจัดเก็บคือเซกเตอร์) โดย*เซกเตอร์* นิยามว่าเป็น 1/4 ของแคชไลน์ (กล่าวคือ 32 ไบต์)

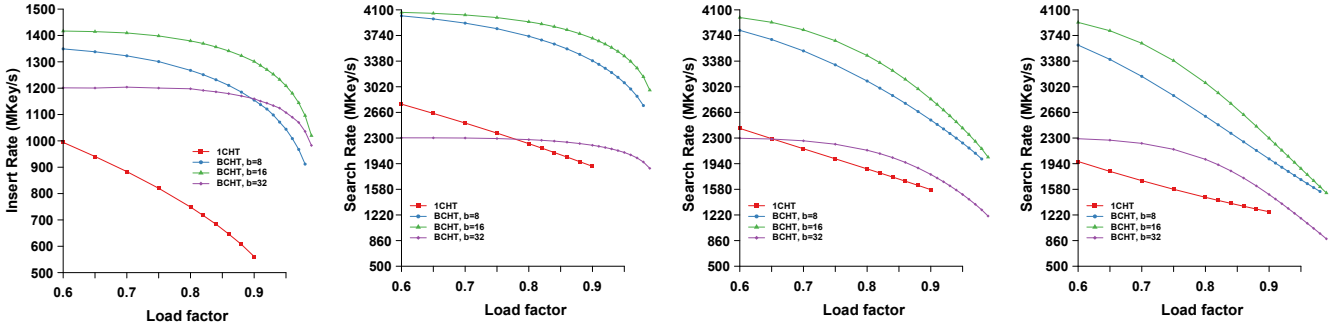
สำหรับการดำเนินการค้นหา ปัจจัยหลักของจำนวนเซกเตอร์ต่อคีย์คือจำนวนบักเก็ตที่การค้นหาต้องอ่านเพื่อหาคีย์ ตัวอย่างเช่น เมื่อ BCHT พบคีย์ในบักเก็ตแรกที่เป็นไปได้ (กล่าวคือ ต้องใช้บักเก็ตเดียวในการประเมินการค้นหา) เราคาดว่าจำนวนเซกเตอร์ต่อคีย์จะเป็นสี่ (12 ในสถานการณ์ที่เลวร้ายที่สุดซึ่งมีสามบักเก็ต) ในทำนองเดียวกัน BP2HT และ IHT จะต้องใช้สี่เซกเตอร์ในสถานการณ์ที่ดีที่สุด และ 8 และ 12 เซกเตอร์ในสถานการณ์ที่เลวร้ายที่สุดตามลำดับ 1CHT ต้องใช้เพียงจำนวนเซกเตอร์ระหว่าง 2 ถึง 8 (สังเกตว่าหน่วยย่อยของการเข้าถึง (access granularity) ระหว่าง DRAM กับแคช L2 คือ 64 ไบต์ กล่าวคือ เมื่อการเข้าถึงแบบไม่รวมกลุ่ม (uncoalesced) อ่านเซกเตอร์เดียว จะมีการถ่ายโอนสองเซกเตอร์)

ในทางกลับกัน การแทรกในตารางแบบแบ่งเป็นบักเก็ตต้องเขียนคู่กลับลงในบักเก็ตโดยใช้การเปรียบเทียบและสลับแบบอะตอมิกอย่างน้อยหนึ่งครั้ง (และในบางกรณีใช้การสลับแบบอะตอมิกหากบักเก็ตเต็ม หรือใน 1CHT) เนื่องจากเราดำเนินการแบบอะตอมิกเหล่านี้กับคู่คีย์–ค่าขนาดเพียง 8 ไบต์ การดำเนินการแบบอะตอมิกจะเพิ่มเซกเตอร์อีกอย่างน้อยหนึ่งเซกเตอร์ เราเห็นผลนี้ทั้งใน IHT และ BCHT ในรูปที่ 15 สังเกตว่าการดำเนินการแบบอะตอมิกเหล่านี้แบ่งออกเป็นสองการดำเนินการ: ขั้นแรก ย้ายเซกเตอร์ที่ต้องการไปยังแคช L2 ตามด้วยการดำเนินการปรับปรุงและเขียนกลับ (writeback) อย่างไรก็ตาม เนื่องจากเราเพิ่งดึงบักเก็ตมาก่อนการดำเนินการแบบอะตอมิกมากกว่า 90% ของเวลา บักเก็ตจึงอยู่ในแคช L2 อยู่แล้ว

ในทางกลับกัน BP2HT แสดงการเพิ่มขึ้นของจำนวนเซกเตอร์ระหว่างการค้นหาและการแทรกมากกว่าหนึ่ง เหตุผลเบื้องหลังคือ BP2HT ต้องโหลดสองบักเก็ตก่อนตัดสินใจว่าจะแทรกคู่ลงในบักเก็ตใด ซึ่งเพิ่มจำนวนเซกเตอร์อีกสี่เซกเตอร์ ส่งผลให้มีเซกเตอร์เพิ่มขึ้นรวมห้าเซกเตอร์เมื่อเทียบกับการค้นหา

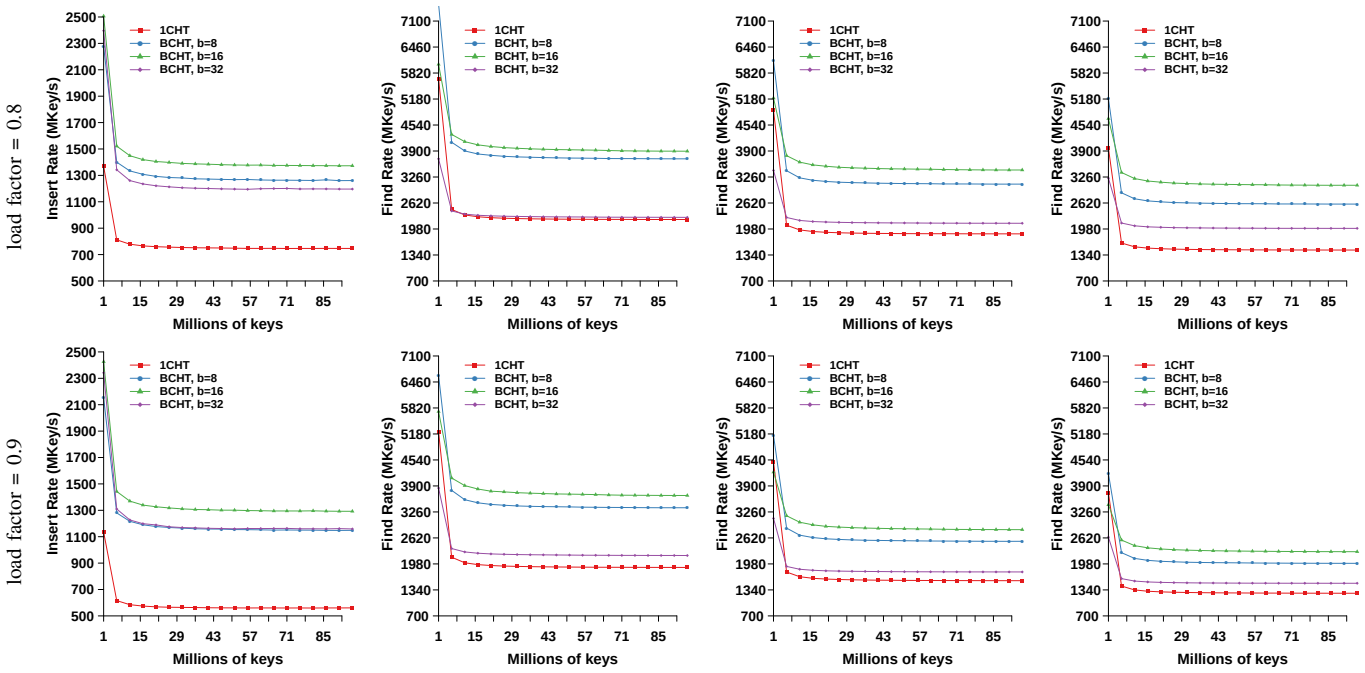
เราขอตั้งข้อสังเกตว่า แม้ในที่นี้เราจะใช้เซกเตอร์เป็นหน่วย แต่การโหลดหนึ่งเซกเตอร์จาก DRAM มีต้นทุนเท่ากับการโหลดแคชไลน์เต็มไลน์ ดังนั้นเซกเตอร์ที่เพิ่มขึ้นหนึ่งเซกเตอร์ (โดยเฉพาะสำหรับการเขียนคู่กลับลงในบักเก็ต) จึงมีต้นทุนเท่ากับการอ่านแคชไลน์ทั้งไลน์

ข้อสรุปของเราจากการวิเคราะห์นี้คือ เราใช้แบนด์วิดท์ DRAM ได้อย่างมีประสิทธิภาพ และปัจจัยหลักที่จำกัดประสิทธิภาพคือจำนวนเซกเตอร์ต่อคีย์ที่การดำเนินการของตารางแฮชต้องโหลด (หรือจัดเก็บ) การวิเคราะห์ของเราในที่นี้สอดคล้องอย่างใกล้ชิดกับผลอัตราการประมวลผลของการแทรก (หรือการค้นหา) ในรูปที่ 12

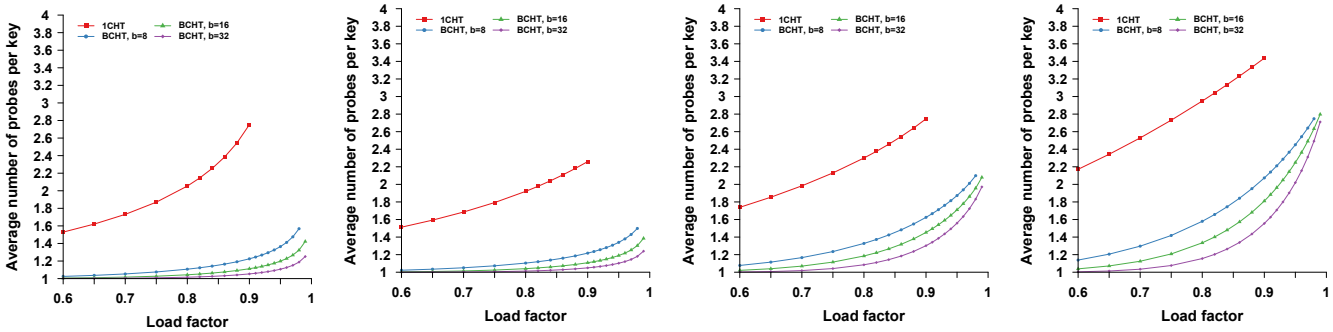


(a) การแทรก (b) การค้นหา (คีย์ที่ค้นหาพบ 100%) (c) การค้นหา (คีย์ที่ค้นหาพบ 50%) (d) การค้นหา (คีย์ที่ค้นหาพบ 0%)

รูปที่ 3: อัตราการแทรกและการค้นหาของ BCHT สำหรับอัตราส่วนการค้นหาแบบพบต่าง ๆ และจำนวนคีย์ 50M คีย์

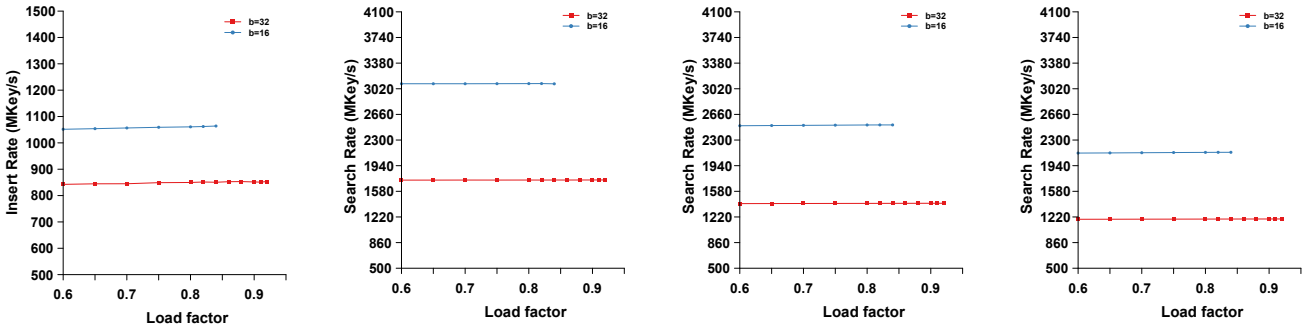


รูปที่ 4: อัตราการประมวลผลของ BCHT สำหรับการแทรก และการค้นหาแบบพบ 100%, 50% และ 0% (จากซ้ายไปขวา)



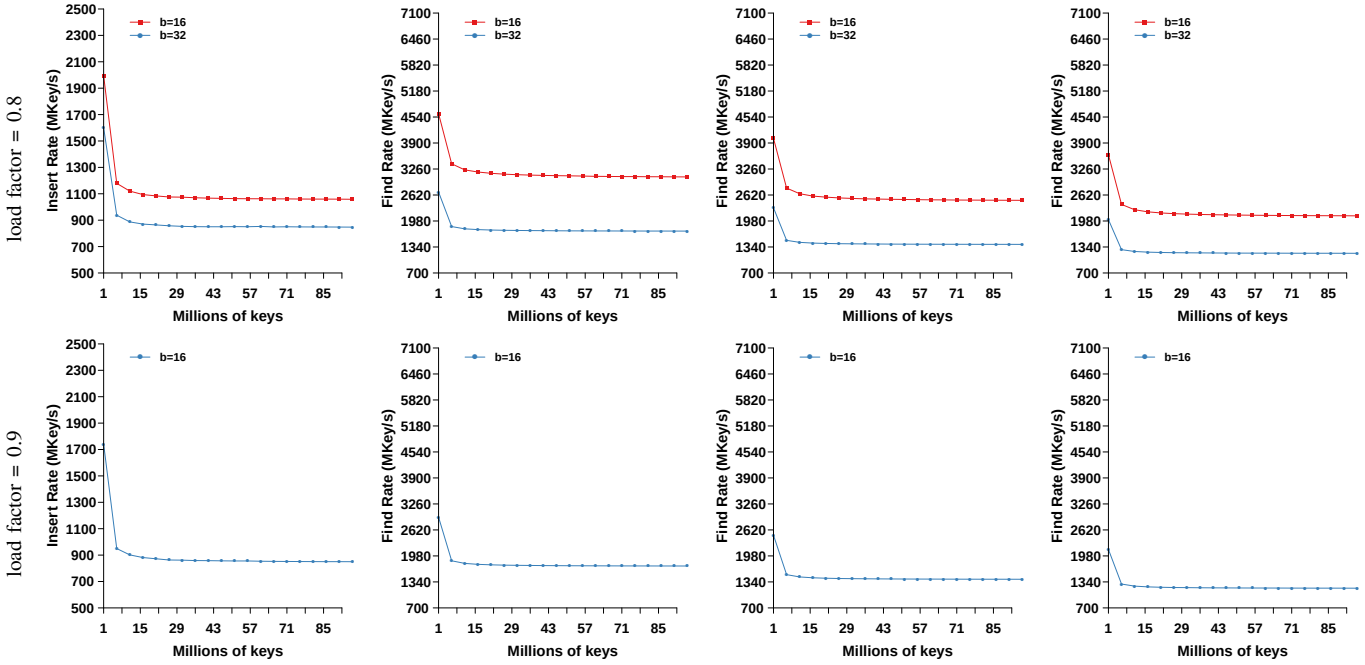
(a) การแทรก (b) การค้นหา (คีย์ที่ค้นหาพบ 100%) (c) การค้นหา (คีย์ที่ค้นหาพบ 50%) (d) การค้นหา (คีย์ที่ค้นหาพบ 0%)

รูปที่ 5: จำนวนการโพรบเฉลี่ยต่อคีย์ของการแทรกและการค้นหาใน BCHT สำหรับ 50M คีย์

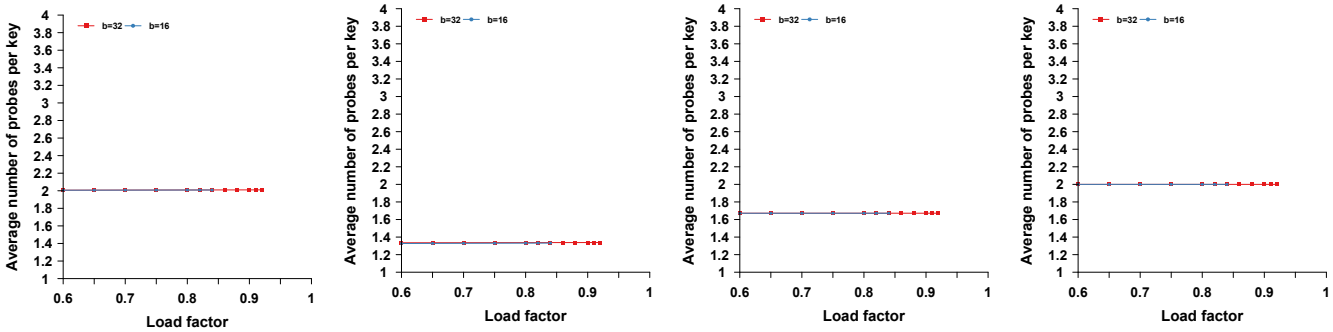


(a) การแทรก (b) การค้นหา (คีย์ที่ค้นหาพบ 100%) (c) การค้นหา (คีย์ที่ค้นหาพบ 50%) (d) การค้นหา (คีย์ที่ค้นหาพบ 0%)

รูปที่ 6: อัตราการแทรกและการค้นหาของ BP2HT สำหรับอัตราส่วนการค้นหาแบบพบต่าง ๆ และจำนวนคีย์ 50M คีย์

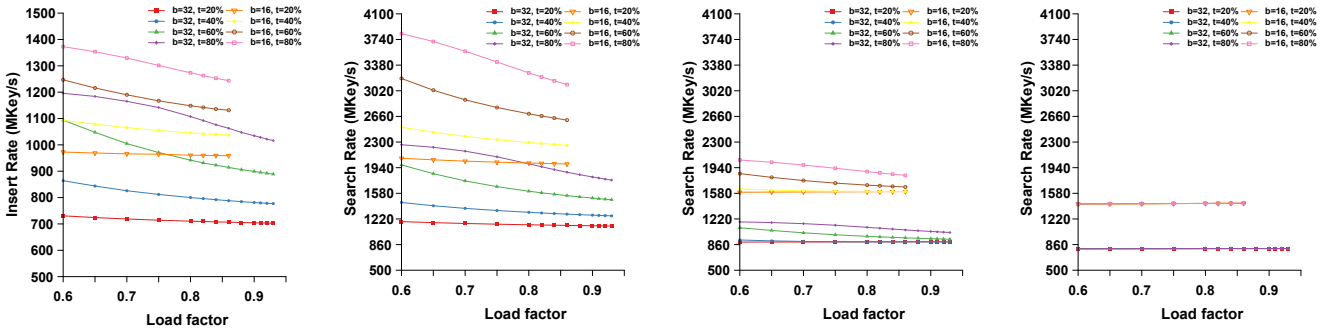


รูปที่ 7: อัตราการประมวลผลของ BP2HT สำหรับการแทรก และการค้นหาแบบพบ 100%, 50% และ 0% (จากซ้ายไปขวา)



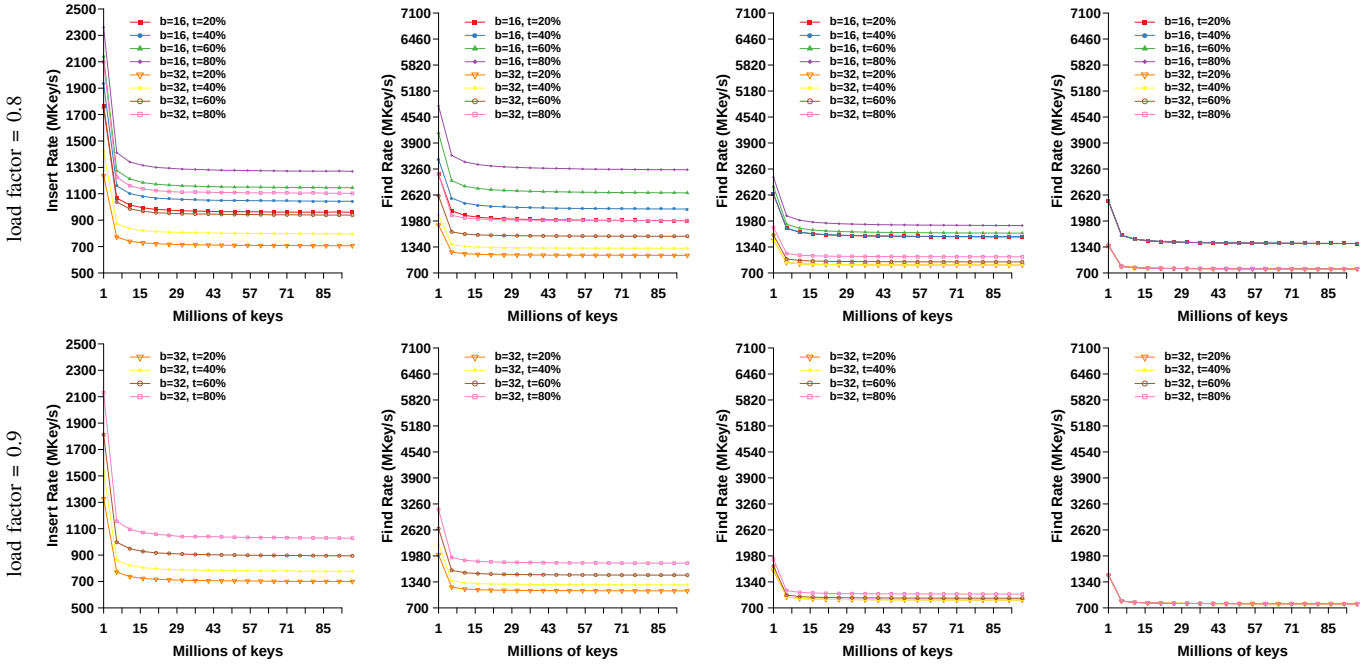
(a) การแทรก (b) การค้นหา (คีย์ที่ค้นหาพบ 100%) (c) การค้นหา (คีย์ที่ค้นหาพบ 50%) (d) การค้นหา (คีย์ที่ค้นหาพบ 0%)

รูปที่ 8: จำนวนการโพรบเฉลี่ยต่อคีย์ของการแทรกและการค้นหาใน BP2HT สำหรับ 50M คีย์

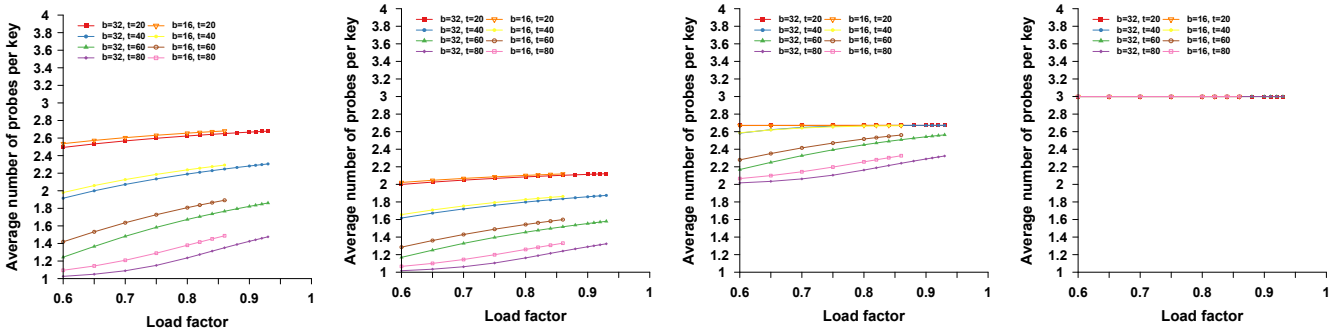


(a) การแทรก (b) การค้นหา (คีย์ที่ค้นหาพบ 100%) (c) การค้นหา (คีย์ที่ค้นหาพบ 50%) (d) การค้นหา (คีย์ที่ค้นหาพบ 0%)

รูปที่ 9: อัตราการแทรกและการค้นหาของ IHT สำหรับอัตราส่วนการค้นหาแบบพบต่าง ๆ และจำนวนคีย์ 50M คีย์

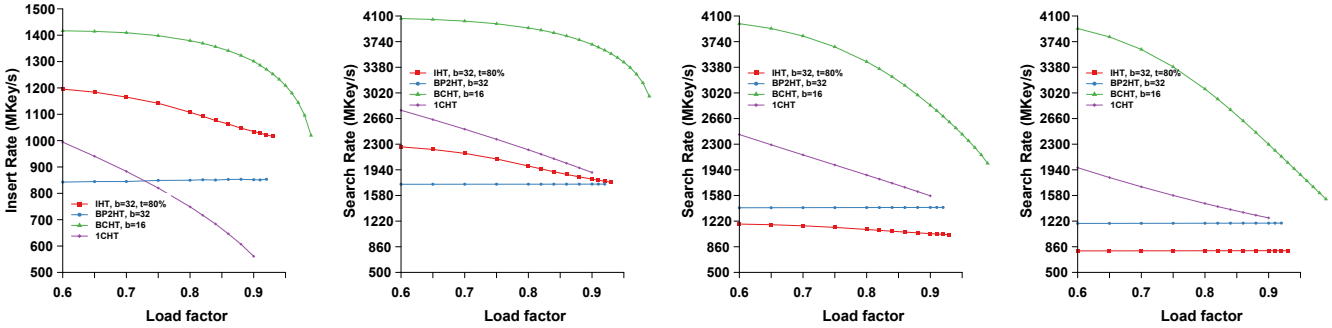


รูปที่ 10: อัตราการประมวลผลของ IHT สำหรับการแทรก และการค้นหาแบบพบ 100%, 50% และ 0% (จากซ้ายไปขวา)

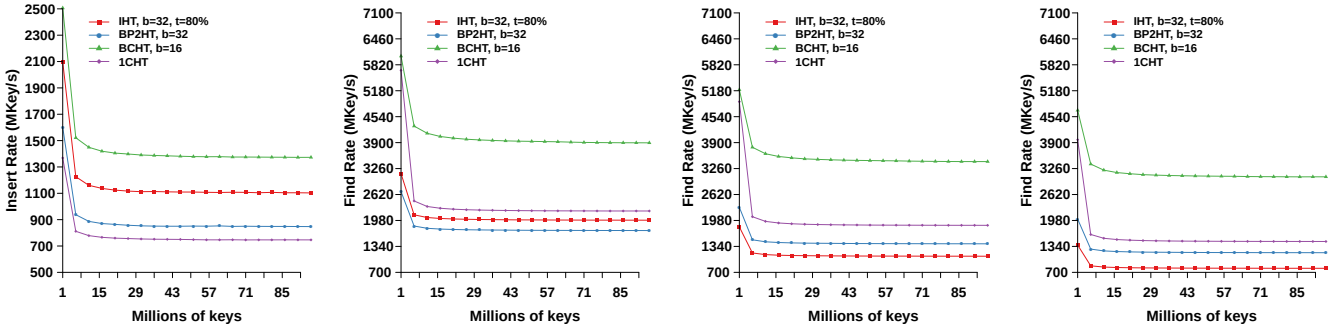


(a) การแทรก (b) การค้นหา (คีย์ที่ค้นหาพบ 100%) (c) การค้นหา (คีย์ที่ค้นหาพบ 50%) (d) การค้นหา (คีย์ที่ค้นหาพบ 0%)

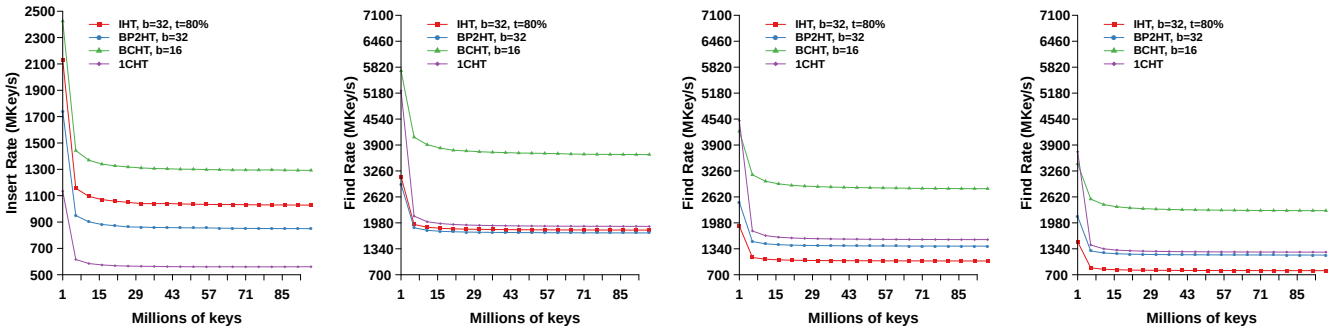
รูปที่ 11: จำนวนการโพรบเฉลี่ยต่อคีย์ของการแทรกและการค้นหาใน IHT สำหรับ 50M คีย์



รูปที่ 12: อัตราการแทรกและการค้นหาเฉลี่ย (การค้นหาแบบพบ 100%, 50%, 0%) สำหรับ load factor ต่าง ๆ และตารางแฮชรูปแบบที่แนะนำ จำนวนคีย์คือ 50M คีย์

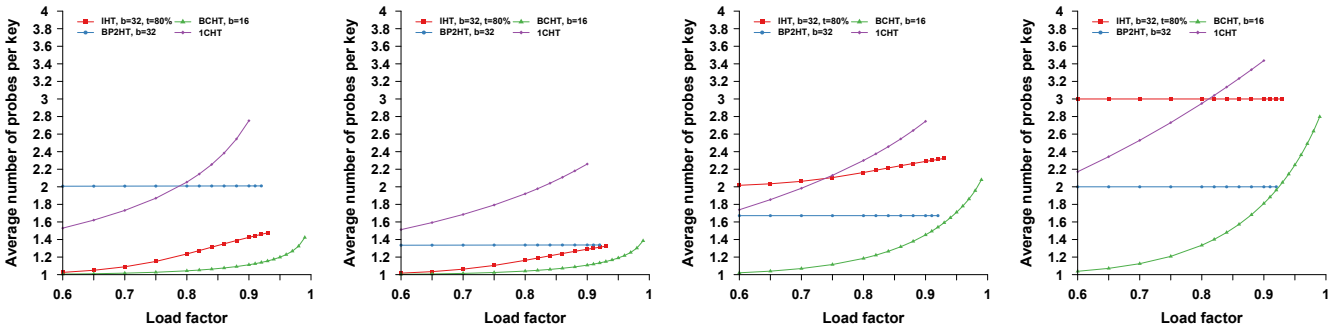


(a) อัตราการแทรก และอัตราการค้นหาสำหรับการค้นหาแบบพบ 100%, 50% และ 0% load factor เท่ากับ 0.8

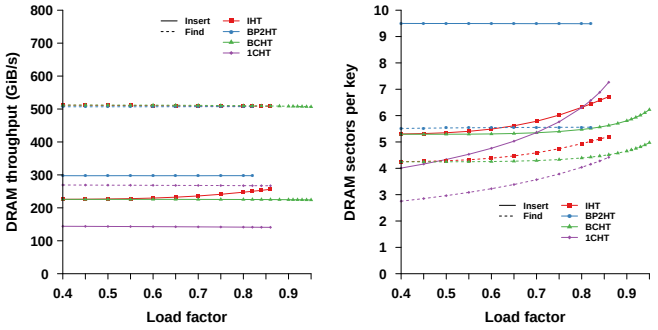


(b) อัตราการแทรก และอัตราการค้นหาสำหรับการค้นหาแบบพบ 100%, 50% และ 0% load factor เท่ากับ 0.9

รูปที่ 13: อัตราการแทรกและการค้นหาเฉลี่ยสำหรับจำนวนคีย์ต่าง ๆ และตารางแฮชรูปแบบที่แนะนำ จำนวนคีย์คือ 50M คีย์



รูปที่ 14: จำนวนการโพรบเฉลี่ยต่อคีย์สำหรับการแทรกและการค้นหา (การค้นหาแบบพบ 100%, 50%, 0%) เมื่อ load factor เปลี่ยนแปลง จำนวนคีย์คือ 50M คีย์



รูปที่ 15: อัตราการประมวลผลที่ทำได้และจำนวนเซกเตอร์ (sector) เฉลี่ยต่อคีย์ (หนึ่งเซกเตอร์คือ 32 ไบต์) สำหรับ IHT, BCHT และ BP2HT (แต่ละแบบใช้บักเก็ตขนาด 16 และ IHT ใช้ค่าขีดเริ่ม 13 คีย์ต่อบักเก็ตหลัก) จำนวนคีย์คือ 200M คีย์ และคีย์ที่ค้นหาทั้งหมดมีอยู่ในตารางแฮช สังเกตความสัมพันธ์ระหว่างจำนวนการโพรบเฉลี่ยต่อคีย์ (รูปที่ 14) กับจำนวนเซกเตอร์เฉลี่ยต่อคีย์ การโพรบหนึ่งครั้งในบักเก็ตขนาด 128 ไบต์เท่ากับการโหลดสี่เซกเตอร์ นอกจากนี้ ให้สังเกตความสัมพันธ์ผกผันระหว่างจำนวนเซกเตอร์เฉลี่ยต่อคีย์กับอัตราการประมวลผลเฉลี่ยของการแทรกหรือการค้นหาในรูปที่ 12

VIII. บทสรุปและงานในอนาคต

จำนวนการโพรบที่การดำเนินการแทรก (หรือการค้นหา) ต้องทำเป็นปัจจัยหลักที่จำกัดประสิทธิภาพของตารางแฮช เมื่อมีการออกแบบและการพัฒนาที่มีประสิทธิภาพ เราได้พัฒนาตารางแฮชแบบสถิตสามรูปแบบบน GPU อย่างมีประสิทธิภาพ โดยความแตกต่างหลักระหว่างตารางแฮชแต่ละรูปแบบคือรูปแบบการโพรบ

สำหรับปัญหาที่ถูกจำกัดด้วยหน่วยความจำ เช่น การสร้างและการค้นหาในตารางแฮช การออกแบบที่มีประสิทธิภาพจะใช้แบนด์วิดท์หน่วยความจำได้อย่างเต็มที่ ตารางแฮชแบบแบ่งเป็นบักเก็ตที่ขนาดบักเก็ตตรงกับแคชไลน์ของฮาร์ดแวร์ทำให้ได้การออกแบบตารางแฮชที่เหมาะกับ GPU ผลการทดลองของเราแสดงให้เห็นว่าการแก้ปัญหาการชนกันด้วย cuckoo hashing มีประสิทธิภาพเหนือกว่าเทคนิคทางเลือกอื่นบน GPU ด้วยจำนวนการโพรบเฉลี่ยประมาณ 1.43 ครั้งต่อคีย์ที่ load factor สูงถึง 0.99 ตารางแฮชคุกคูแบบแบ่งเป็นบักเก็ตจึงเป็นทางเลือกที่ดีที่สุดโดยรวมในบรรดาตารางแฮชทั้งหมดที่เราศึกษา

ในงานอนาคต เราต้องการกลับมาศึกษาเทคนิค iceberg hashing อีกครั้ง โดยเฉพาะอย่างยิ่ง เราต้องการบรรลุ load factor สูงเมื่อใช้ขนาดบักเก็ตที่เหมาะสมที่สุดสำหรับ GPU (กล่าวคือ $b = 16$) โดยศึกษารูปแบบการโพรบต่าง ๆ หลังจากบักเก็ตถึงค่าขีดเริ่มแล้ว นอกจากนี้ เพื่อปรับปรุงประสิทธิภาพของการค้นหาแบบไม่พบ เราสามารถใช้ quotient filter ที่เก็บข้อมูลเกี่ยวกับคีย์ที่ไม่สามารถใส่ลงในบักเก็ตหลักได้

กิตติกรรมประกาศ

ขอขอบคุณ Martin Dietzfelbinger, Lars Nyland, Nicolas Iskos และ Alex Conway สำหรับข้อเสนอแนะทางเทคนิคที่เป็นประโยชน์ ขอขอบคุณ Aleksy Bałaziński ที่ชี้ข้อผิดพลาดในรายการโค้ด

คณะผู้เขียนขอขอบคุณการสนับสนุนการวิจัยจาก National Science Foundation (ทุนเลขที่ # CCF-1637442 และ # OAC-1740333), DARPA (ทุน AFRL เลขที่ # FA8650-18-2-7835 และ # HR0011-18-3-0007), Adobe Data Science Research Award, NVIDIA AI Laboratory และอุปกรณ์ที่ได้รับบริจาคจาก NVIDIA

งานนี้อิงจากการวิจัยที่ได้รับการสนับสนุนจาก Air Force Research Lab (AFRL) และ Defense Advanced Research Projects Agency (DARPA) รัฐบาลสหรัฐอเมริกามีสิทธิ์ทำซ้ำและแจกจ่ายสำเนาเพื่อวัตถุประสงค์ของรัฐบาล โดยไม่คำนึงถึงข้อความแสดงลิขสิทธิ์ใด ๆ ในเอกสารนี้ ทัศนะและข้อสรุปที่ปรากฏในเอกสารนี้เป็นของผู้เขียน และไม่ควรตีความว่าเป็นตัวแทนของนโยบายหรือการรับรองอย่างเป็นทางการ ไม่ว่าโดยชัดแจ้งหรือโดยนัย ของ Air Force Research Lab (AFRL) และ Defense Advanced Research Projects Agency (DARPA) หรือรัฐบาลสหรัฐอเมริกา

เอกสารอ้างอิง

[1] D. A. Alcantara, A. Sharf, F. Abbasinejad, S. Sengupta, M. Mitzenmacher, J. D. Owens, and N. Amenta, "Real-time parallel hashing on the GPU," *ACM Transactions on Graphics*, vol. 28, no. 5, pp. 154:1–154:9, Dec. 2009. [Online]. Available: https://escholarship.org/uc/item/445536d6

[2] D. A. Alcantara, V. Volkov, S. Sengupta, M. Mitzenmacher, J. D. Owens, and N. Amenta, "Building an efficient hash table on the GPU," in *GPU Computing Gems*, W. W. Hwu, Ed. Morgan Kaufmann, Oct. 2011, vol. 2, ch. 4, pp. 39–53.

[3] S. Ashkiani, M. Farach-Colton, and J. D. Owens, "A dynamic hash table for the GPU," in *Proceedings of the 32nd IEEE International Parallel and Distributed Processing Symposium*, ser. IPDPS 2018, May 2018, pp. 419–429. [Online]. Available: https://escholarship.org/uc/item/2p48q0zg

[4] Y. Azar, A. Z. Broder, A. R. Karlin, and E. Upfal, "Balanced allocations," *SIAM Journal on Computing*, vol. 29, no. 1, pp. 180–200, Jan. 1999.

[5] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proceedings of the 2009 ACM/IEEE Conference on Supercomputing*, ser. SC '09, Nov. 2009, pp. 18:1–18:11.

[6] A. D. Breslow, D. P. Zhang, J. L. Greathouse, N. Jayasena, and D. M. Tullsen, "Horton tables: Fast hash tables for in-memory data-intensive computing," in *Proceedings of the 2016 USENIX Conference on Usenix Annual Technical Conference*, ser. USENIX ATC '16. USA: USENIX Association, Jun. 2016, pp. 281–294.

[7] M. Dietzfelbinger, A. Goerdt, M. Mitzenmacher, A. Montanari, R. Pagh, and M. Rink, "Tight thresholds for cuckoo hashing via XORSAT," in *International Colloquium on Automata, Languages and Programming*, 2010, pp. 213–225.

[8] Ú. Erlingsson, M. Manasse, and F. McSherry, "A cool and practical alternative to traditional hash tables," in *Proceedings of the 7th Workshop on Distributed Data and Structures*, ser. WDAS '06, Jan. 2006.

[9] D. Fotakis, R. Pagh, P. Sanders, and P. Spirakis, "Space efficient hash tables with worst case constant access time," *Theory of Computing Systems*, vol. 38, no. 2, pp. 229–248, 2005.

[10] M. Harris, J. D. Owens, S. Sengupta, Y. Zhang, and A. Davidson, "CUDPP: CUDA data parallel primitives library," 2009–2017, http://cudpp.github.io/.

[11] Z. Jia, M. Maggioni, B. Staiger, and D. P. Scarpazza, "Dissecting the NVIDIA Volta GPU architecture via microbenchmarking," *CoRR*, Apr. 2018, arXiv:cs.DC/1804.06826v1.

[12] D. Jünger, R. Kobus, A. Müller, C. Hundt, K. Xu, W. Liu, and B. Schmidt, "WarpCore: A library for fast hash tables on GPUs," 2020.

[13] D. Jütnger, C. Hundt, and B. Schmidt, "WarpDrive: Massively parallel hashing on multi-GPU nodes," in *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2018, pp. 441–450.

[14] B. Lessley and H. Childs, "Data-parallel hashing techniques for GPU architectures," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 1, pp. 237–250, Jan. 2020.

[15] Y. Li, Q. Zhu, Z. Lyu, Z. Huang, and J. Sun, "DyCuckoo: Dynamic hash tables on GPUs," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE Computer Society, Apr. 2021, pp. 744–755. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/ICDE51399.2021.00070

[16] G. Marsaglia, "Xorshift rngs," *Journal of Statistical Software*, vol. 8, no. 14, pp. 1–6, 2003.

[17] M. Mitzenmacher, A. W. Richa, and R. Sitaraman, "The power of two random choices: A survey of techniques and results," in *Handbook of Randomized Computing*, ser. Combinatorial optimization, S. Rajasekaran, P. M. Pardalos, J. H. Reif, and J. Rolim, Eds. Kluwer Academic Publishers, Jun. 2001, vol. I, pp. 255–312.

[18] Omitted for Anonymity, "Dynamic balls-and-bins and iceberg hashing," *in progress*, 2021.

[19] R. Pagh and F. F. Rodler, "Cuckoo hashing," in *9th Annual European Symposium on Algorithms*, ser. Lecture Notes in Computer Science, vol. 2161. Springer, Aug. 2001, pp. 121–133.

[20] R. Panigrahy, "Efficient hashing with lookups in two memory accesses," in *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, ser. SODA '05. USA: Society for Industrial and Applied Mathematics, 2005, p. 830–839.

[21] S. Walzer, "Random hypergraphs for hashing-based data structures," Ph.D. dissertation, Technische Universität Ilmenau, Ilmenau, Nov 2020, dissertation, Technische Universität Ilmenau, 2020. [Online]. Available: https://www.db-thueringen.de/receive/dbt_mods_00047127

[22] K. Zhang, K. Wang, Y. Yuan, L. Guo, R. Lee, and X. Zhang, "Mega-KV: A case for GPUs to maximize the throughput of in-memory key-value stores," *Proceedings of the VLDB Endowment*, vol. 8, no. 11, pp. 1226–1237, Jul. 2015.